LL984 Editor

Reference Manual
LL984 Specifics

09/2017

EIO0000000549.10

www.schneider-electric.com

Schneider Electric

The information provided in this documentation contains general descriptions and/or technical characteristics of the performance of the products contained herein. This documentation is not intended as a substitute for and is not to be used for determining suitability or reliability of these products for specific user applications. It is the duty of any such user or integrator to perform the appropriate and complete risk analysis, evaluation and testing of the products with respect to the relevant specific application or use thereof. Neither Schneider Electric nor any of its affiliates or subsidiaries shall be responsible or liable for misuse of the information contained herein. If you have any suggestions for improvements or amendments or have found errors in this publication, please notify us.

You agree not to reproduce, other than for your own personal, noncommercial use, all or part of this document on any medium whatsoever without permission of Schneider Electric, given in writing. You also agree not to establish any hypertext links to this document or its content. Schneider Electric does not grant any right or license for the personal and noncommercial use of the document or its content, except for a non-exclusive license to consult it on an "as is" basis, at your own risk. All other rights are reserved.

All pertinent state, regional, and local safety regulations must be observed when installing and using this product. For reasons of safety and to help ensure compliance with documented system data, only the manufacturer should perform repairs to components.

When devices are used for applications with technical safety requirements, the relevant instructions must be followed.

Failure to use Schneider Electric software or approved software with our hardware products may result in injury, harm, or improper operating results.

Failure to observe this information can result in injury or equipment damage.

© 2017 Schneider Electric. All Rights Reserved.

Table of Contents

	Safety Information	7
	About the Book	11
Part I	**Data Editor**	13
Chapter 1	**General Functions of the Data Editor**	15
	Configuring the Data Editor columns	15
Part II	**Programming**	19
Chapter 2	**General Functions of the LL984 Editor**	21
	General Functions and Layout of the LL984 Editor	22
	Toolbar of the LL984 Editor	24
	General Restrictions	27
Chapter 3	**Accessing Tasks and Sections**	33
	Creating LL984 Segments and Networks	34
	Modifying LL984 Segment Properties	38
	Modifying LL984 Network Properties	40
	Scheduling LL984 Segments and Network	42
	Deleting, Cutting, Copying and Pasting LL984 Segments and Network	43
Chapter 4	**Unity Pro Software Options**	45
4.1	Project Settings	46
	Program	46
4.2	Options	51
	Data and Languages	51
Chapter 5	**Common Functions for Graphical Languages**	53
	Using Bookmarks in Graphical Languages Sections	53
Chapter 6	**LL984 Editor**	59
6.1	Creating a Program with LL984 Programming Language	60
	Project Settings for LL984 Programs	61
	Structure of an LL984 Program	62
	Creating an LL984 Program	65
	Syntax and Semantics Check During Programming	68
	Network Display	71
	Navigating with the Keyboard	72

Selecting Objects 74
Deleting, Cutting, Copying, Pasting and Moving Objects 76
Displaying the Properties 80
Inserting and Deleting Rows/Columns 81
6.2 Editing Contacts 83
Selecting Contacts 84
Placing Contacts 86
Properties Dialog for Contacts 90
6.3 Editing Coils 93
Selecting Coils 94
Placing Coils 96
Property Dialog for Coils 99
6.4 Editing Function Blocks 101
Editing Function Blocks 102
Using Mnemonics 105
6.5 Editing Links 106
Selecting Links 107
Combining Links 108
Placing Links 109
Editing Links 111
6.6 Editing Equation Network Blocks 112
Editing Equation Network Blocks 113
Editing the ST Assignment Expression 115
6.7 Assigning Variables to LL984 Objects 119
Assigning Variables to LL984 Objects 119
6.8 Calling Subroutines 123
Calling Subroutines 123
6.9 Entering Comments 125
Entering Comments 125
Chapter 7 Programming Languages Data Selection Dialog 127
Data Properties 127
Chapter 8 Search/Replace Tool 129
Overview of the Search / Replace tool 130
General Features for Search/Replace 132
Searching for Data 133
Replacing Data 134
Search Toolbar 135

Data Search Dialog 136
Cross-References (Search Results) 138
Cross-References (Replace) 141
Tracing Variables / Addresses 143
Part III Debugging and Adjustment 145
Chapter 9 Viewing and Adjusting Variables 147
Force Mode 148
Instruction Editor 154
Appendices 157
Appendix A LL984 operating modes 159
LL984 operating modes 159
Index 161

Safety Information

Important Information

NOTICE

Read these instructions carefully, and look at the equipment to become familiar with the device before trying to install, operate, service, or maintain it. The following special messages may appear throughout this documentation or on the equipment to warn of potential hazards or to call attention to information that clarifies or simplifies a procedure.



The addition of this symbol to a “Danger” or “Warning” safety label indicates that an electrical hazard exists which will result in personal injury if the instructions are not followed.



This is the safety alert symbol. It is used to alert you to potential personal injury hazards. Obey all safety messages that follow this symbol to avoid possible injury or death.

⚠ DANGER

DANGER indicates a hazardous situation which, if not avoided, **will result in** death or serious injury.

⚠ WARNING

WARNING indicates a hazardous situation which, if not avoided, **could result in** death or serious injury.

⚠ CAUTION

CAUTION indicates a hazardous situation which, if not avoided, **could result** in minor or moderate injury.

NOTICE

NOTICE is used to address practices not related to physical injury.

PLEASE NOTE

Electrical equipment should be installed, operated, serviced, and maintained only by qualified personnel. No responsibility is assumed by Schneider Electric for any consequences arising out of the use of this material.

A qualified person is one who has skills and knowledge related to the construction and operation of electrical equipment and its installation, and has received safety training to recognize and avoid the hazards involved.

BEFORE YOU BEGIN

Do not use this product on machinery lacking effective point-of-operation guarding. Lack of effective point-of-operation guarding on a machine can result in serious injury to the operator of that machine.

⚠ WARNING

UNGUARDED EQUIPMENT

- Do not use this software and related automation equipment on equipment which does not have point-of-operation protection.
- Do not reach into machinery during operation.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

This automation equipment and related software is used to control a variety of industrial processes. The type or model of automation equipment suitable for each application will vary depending on factors such as the control function required, degree of protection required, production methods, unusual conditions, government regulations, etc. In some applications, more than one processor may be required, as when backup redundancy is needed.

Only you, the user, machine builder or system integrator can be aware of all the conditions and factors present during setup, operation, and maintenance of the machine and, therefore, can determine the automation equipment and the related safeties and interlocks which can be properly used. When selecting automation and control equipment and related software for a particular application, you should refer to the applicable local and national standards and regulations. The National Safety Council's Accident Prevention Manual (nationally recognized in the United States of America) also provides much useful information.

In some applications, such as packaging machinery, additional operator protection such as point-of-operation guarding must be provided. This is necessary if the operator's hands and other parts of the body are free to enter the pinch points or other hazardous areas and serious injury can occur. Software products alone cannot protect an operator from injury. For this reason the software cannot be substituted for or take the place of point-of-operation protection.

Ensure that appropriate safeties and mechanical/electrical interlocks related to point-of-operation protection have been installed and are operational before placing the equipment into service. All interlocks and safeties related to point-of-operation protection must be coordinated with the related automation equipment and software programming.

NOTE: Coordination of safeties and mechanical/electrical interlocks for point-of-operation protection is outside the scope of the Function Block Library, System User Guide, or other implementation referenced in this documentation.

START-UP AND TEST

Before using electrical control and automation equipment for regular operation after installation, the system should be given a start-up test by qualified personnel to verify correct operation of the equipment. It is important that arrangements for such a check be made and that enough time is allowed to perform complete and satisfactory testing.

⚠ WARNING

EQUIPMENT OPERATION HAZARD

- Verify that all installation and set up procedures have been completed.
- Before operational tests are performed, remove all blocks or other temporary holding means used for shipment from all component devices.
- Remove tools, meters, and debris from equipment.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Follow all start-up tests recommended in the equipment documentation. Store all equipment documentation for future references.

Software testing must be done in both simulated and real environments.

Verify that the completed system is free from all short circuits and temporary grounds that are not installed according to local regulations (according to the National Electrical Code in the U.S.A, for instance). If high-potential voltage testing is necessary, follow recommendations in equipment documentation to prevent accidental equipment damage.

Before energizing equipment:

- Remove tools, meters, and debris from equipment.
- Close the equipment enclosure door.
- Remove all temporary grounds from incoming power lines.
- Perform all start-up tests recommended by the manufacturer.

OPERATION AND ADJUSTMENTS

The following precautions are from the NEMA Standards Publication ICS 7.1-1995 (English version prevails):

- Regardless of the care exercised in the design and manufacture of equipment or in the selection and ratings of components, there are hazards that can be encountered if such equipment is improperly operated.
- It is sometimes possible to misadjust the equipment and thus produce unsatisfactory or unsafe operation. Always use the manufacturer's instructions as a guide for functional adjustments. Personnel who have access to these adjustments should be familiar with the equipment manufacturer's instructions and the machinery used with the electrical equipment.
- Only those operational adjustments actually required by the operator should be accessible to the operator. Access to other controls should be restricted to prevent unauthorized changes in operating characteristics.

About the Book

At a Glance

Document Scope

This documentation describes the specifics of Unity Pro with LL984 Editor.

Validity Note

This documentation is valid for Unity Pro 13.0 or later.

For standard features refer to the *Unity Pro, Operating Modes Manual*.

NOTE:

- With Unity Pro 6.0 the LL984 language is supported for Quantum PLCs (but not for Quantum Safety PLCs).
- With Unity Pro 6.1 the LL984 language is supported for Modicon M340 PLCs with Modicon M340 firmware 2.4 or later.

Related Documents

Title of Documentation	Reference Number
Unity Pro, UnityLL984, Block Library	EIO0000000550 (English), EIO0000000807 (French), EIO0000000808 (German), EIO0000000809 (Italian), EIO0000000810 (Spanish), EIO0000000811 (Chinese)
Unity Pro, Operating Modes	33003101 (English), 33003102 (French), 33003103 (German), 33003104 (Spanish), 33003696 (Italian), 33003697 (Chinese)
Unity Pro, Program Languages and Structure, Reference Manual	35006144 (English), 35006145 (French), 35006146 (German), 35013361 (Italian), 35006147 (Spanish), 35013362 (Chinese)

Title of Documentation	Reference Number
Unity Pro, System Bits and Words, Reference Manual	EIO0000002135 (English), EIO0000002136 (French), EIO0000002137 (German), EIO0000002138 (Italian), EIO0000002139 (Spanish), EIO0000002140 (Chinese)

You can download these technical publications and other technical information from our website at http://www.schneider-electric.com/en/download

Part I
Data Editor

Chapter 1
General Functions of the Data Editor

Configuring the Data Editor columns

Introduction

In every **Data Editor** tab you can configure the column display (a default configuration is suggested). This lets you:

- Define the columns you want to display.
- Define the sequence for the columns shown.

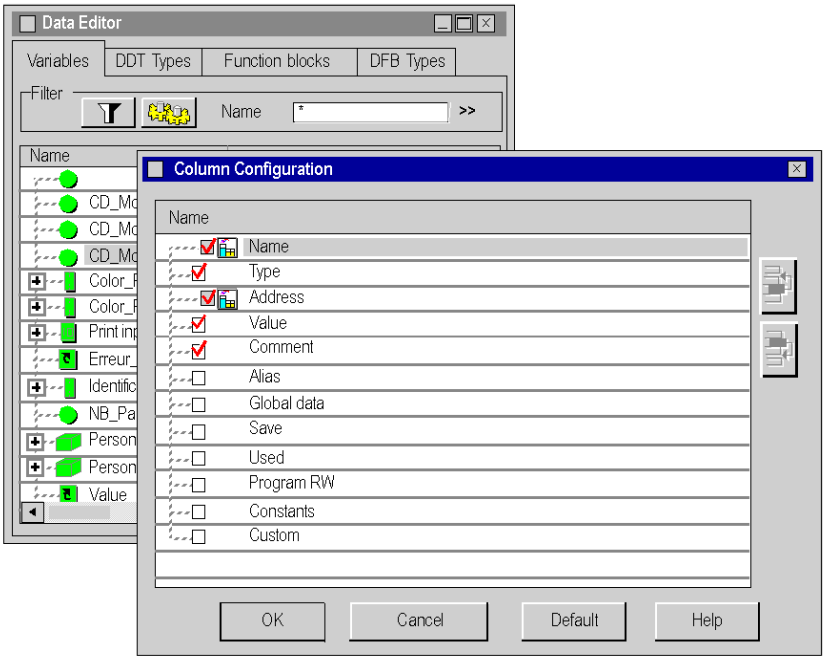
Columns Available

The following table shows the different columns available in the **Data Editor**:

Column Name		Description
Name		Name of the instance or data type.
Type		Data type. By default this attribute is checked.
Address		Address in the controller for referencing the variables. By default this attribute is checked.
Value		Initial value for the variables. By default this attribute is checked.
Comment		Comment for the variables. By default this attribute is checked.
Custom		Free text for use of an external tool or for special user requirements.
Time stamping		Select if a variable is time stamped or not and the kind of time stamping, see the System time stamping variables *(see System Time Stamping, User Guide)* or Applicative time stamping variables *(see Applicative Time Stamping with Unity Pro, User Guide)*.
+	**Source**	Gives the source of the time stamping.
	TS ID	Gives the ID of the time stamped variable.
R/W Rights of Referenced Variable		Access rights of a reference type instance, used when the reference dereferences a read only variable, see the Reference Access rights *(see Unity Pro, Program Languages and Structure, Reference Manual)*. By default this attribute is checked.
Exchange On STBY		Only for M580 Hotstandby systems *(see Modicon M580 Hot Standby, System Planning Guide for, Frequently Used Architectures)*, if checked, the variable or the function block instance is automatically exchanged between Primary PLC and Standby PLC. By default this attribute is checked.
+	**Task**	Give the task in which a Variable, EFB instance, or DFB instance is used.

Column Name		Description
Retain		If checked, the variable, EFB instance, or DFB instance values are automatically stored in the backup memory, if not checked the values are lost on power cut.
no.		Order number of a function block.
984 Address		LL984 address of the variable (%Mx = 00000x, %Ix = 10000x, %IWx = 30000x, %MWx = 40000x) This column is only available for Quantum PLCs (but not for Quantum Safety PLCs).
Alias		Name selected by the user which makes it possible to change an IODDT, DDT, or an Array element to a name defined by the manufacturer or the user. By default this attribute is checked.
Alias of		Complete path of the variable subfield on which user sets Alias attribute. By default this attribute is checked.
Global data		Shows if the variable is a variable that was swapped between different stations via the network.
+	**Group**	Number of the group (the variables) in which the variable is transported.
	ID	Number for the unique display of a variable in the group (the variables).
Save		Define if the variable accepts the mechanism for updating initial values with current values.
Used		Number of times the variables are used in the program and in other parts of the project (animation tables, user windows).
RW program		Access rights of the variable, read only or read and write by the program.
Constant		Kind of variable, the variable is a constant or not.
Owner		Only for **Device DDT**, it displays the address of the modules which owns this variable.
Descriptor x		Descriptors (1 to 9) of variables. **Descriptor 1** is displayed after selecting this column in the **Column Configuration** dialog. **Descriptors 2** to **9** can be displayed using the **+** sign in the **Descriptor** column header.
Version		Type version.
Bit rank		Gives the bit rank of the variable in its structure type.
HMI variable		When **General Project Settings → Data Dictionary → Only HMI variables** is checked, this reduces memory usage. By default this attribute is checked.
Scanned Variables		Attribute that you can use to tag a variable to be a scanned variable by the PLC, through Ethernet for instance.

Defining Columns

Step	Action
1	Select the desired tab in the **Data Editor**.
2	Via the context menu, open the **Column Configuration** dialog using **Customize Columns.** The following dialog appears. 
3	Check the check boxes that correspond to the columns you wish to display.
4	Confirm the changes using the command **OK.**

NOTE: The **Name** and **Type** columns are required.

Description of the buttons

Button	Description
OK	Confirms the configuration and exits the dialog.
Cancel	Discards the changes and exits the dialog.
Default	Restores the default configuration.
Help	Opens the **Help** of the **Column Configuration** dialog.

Changing the Columns Sequence

Step	Action
1	Select the desired tab in the **Data Editor**.
2	Via the context menu, open the **Column Configuration** dialog using **Customize Columns**.
3	Place the cursor on the chosen attribute and move it to the desired location by clicking the two buttons to the right of the window.
4	Confirm by clicking **OK**.

NOTE: The first column must always correspond with the attribute **Name**.

Part II
Programming

Overview

This part describes the specifics of the LL984 Editor.

For standard features please refer to *Programming (see Unity Pro, Operating Modes)*.

What Is in This Part?

This part contains the following chapters:

Chapter	Chapter Name	Page
2	General Functions of the LL984 Editor	21
3	Accessing Tasks and Sections	33
4	Unity Pro Software Options	45
5	Common Functions for Graphical Languages	53
6	LL984 Editor	59
7	Programming Languages Data Selection Dialog	127
8	Search/Replace Tool	129

Chapter 2
General Functions of the LL984 Editor

Overview

This chapter describes the general functions of the LL984 Editor.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
General Functions and Layout of the LL984 Editor	22
Toolbar of the LL984 Editor	24
General Restrictions	27

General Functions and Layout of the LL984 Editor

Introduction

Each LL984 network displayed in the LL984 Editor contains a matrix of 7 rows and 11 columns, where you can place contacts, coils, function blocks etc.

Launching the LL984 Editor

Double-click a LL984 network in the **Program** section of the **Structural View** of the **Project Browser**.

A new LL984 Editor window is opened and the network is displayed there.

LL984 Editor windows can be maximized, so that only one editor window is visible at a time.

Displaying the Network Name

If the editor window is minimized, the name of an LL984 network is displayed in the title bar of the editor.

If the editor window is maximized, the name of an LL984 network is displayed in the title bar of the Unity Pro main window.

Switching Between Networks

Using **Ctrl+Alt+Page Down** / **Ctrl+Alt+Page Up** you can switch to the next/previous section, which is then displayed in the same editor window.

This does not modify the current selection in **Project Browser**, where still the first section is highlighted which was opened by double-click.

If the last/first row of a network is already visible, simple **Page Down/Page Up** displays the next/previous network.

Using **Page Down** in the last network of a segment, if the last row is already visible, will display a message box asking you to create a new network.

Please also refer to Navigating with the Keyboard *(see page 72)*.

Only 1 Window for Open Commands

Under **Tools → Options → Data and Languages → Languages → LL984** you can activate/deactivate the **Reuse 984 editor window when network is opened** option.

If this check box is checked only 1 window is used for **Open** commands on LL984 networks.

If this check box is not checked, for each **Open** command on an LL984 network, a new window is opened.

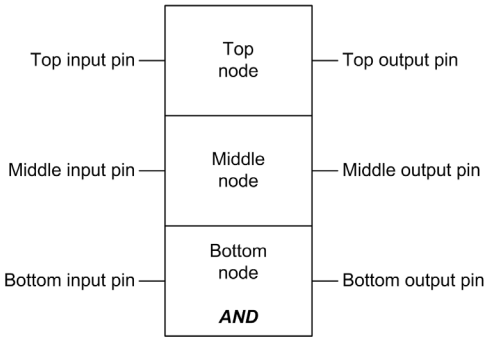
Please refer to Options *(see page 51)*.

Zoom

Use the buttons **Zoom In**, **Zoom Out** and **Zoom to Fit** from the Unity Pro menu bar or the commands under **View → Zoom**.

Displaying the Function Blocks

Example `AND`



The function blocks of the UnityLL984 Block Library are displayed

- 1 cell wide and 1 to 3 cells high
- with boolean pins (inputs/outputs) on the left and the right side of the function block
- with parameters (nodes) in the middle of the block (top, middle and bottom node)
  Not assigned nodes are displayed as `?????`.
- with block type name at the bottom of the block without prefix (for example `L9_AND` is displayed as AND)
  In other tools and editors (like Type Library Browser, Data Editor etc.) the blocks are displayed with their prefix (L9_AND).
- no pin names are shown
- the left and the right side of DFBs are displayed as double lines
- function blocks with detected analyze errors are displayed in blue

If the cursor is placed over a not assigned pin, a tooltip is displayed with pin name and comment. If a pin is assigned, information about the assigned variable is displayed (refer to *Network Display (see page 71)*).

For detailed information please refer to the function block descriptions *(see Unity Pro, UnityLL984, Block Library)*.

Using the Toolbar

Please refer to Toolbar of the LL984 Editor *(see page 24)*.

Toolbar of the LL984 Editor

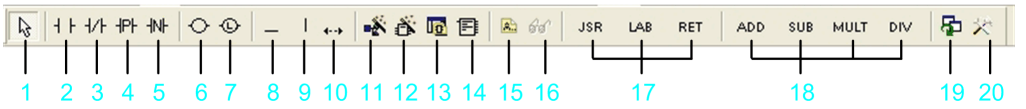
Introduction

The LL984 Editor provides a toolbar at the top of the editor window which contains an arrow button (**Select Mode**), the most frequent LL984 objects, buttons to open special function editor tools, function bocks for subroutines and an assortment of frequently used functions blocks for LL984 networks.

Additionally the toolbar provides a button to open the **LL984 Instruction Bar**.

NOTE: The last selected toolbar button remains selected until the **Esc** key is pressed or the arrow button is selected.

Representation



Elements of the Toolbar

Number	Description
1	Select Mode (Esc) *(see page 74)*
2	Normally open contact (F3) *(see page 84)*
3	Normally closed contact (Shift+F3) *(see page 84)*
4	Positive transition-sensing contact (Ctrl+F3) *(see page 84)*
5	Negative transition-sensing contact (Ctrl+Shift+F3) *(see page 84)*
6	Coil (F5) *(see page 94)*
7	Latched coil (Shift+F5) *(see page 94)*
8	Boolean Connection (F7) *(see page 107)*
9	Vertical Connection (Shift+F7) *(see page 107)*
10	Boolean Link (Alt+F6) *(see page 107)*
11	Data Selection (Ctrl+D) *(see Unity Pro, Operating Modes)*
12	FFB Input Assistant (Ctrl+I) *(see Unity Pro, Operating Modes)*
13	Type Library Browser (Alt+3) *(see Unity Pro, Operating Modes)*
14	Equation Network block *(see page 113)*
15	Comment (F8) *(see page 125)*
16	Inspect Window (F9) *(see Unity Pro, Operating Modes)*

Number	Description
17	Subroutine buttons (JSR, LAB, RET) *(see page 123)* ● JSR (Jump Subroutine) ● LAB (Label the starting point of a Subroutine) ● RET (Return from Subroutine)
18	ADD, SUB, MUL, DIV Here you can directly select frequently used function blocks `L9_ADD`, `L9_SUB`, `L9_MUL`, `L9_DIV`.
19	Change View Mode (Ctrl+W) *(see page 71)*
20	LL984 Instruction Bar (see section below.

Customizing the Toolbar

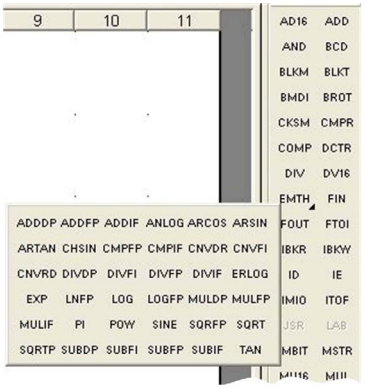
To customize the toolbar open the **Customize** dialog via **Tools → Customize → Toolbars** and for example drag a function block button from the **LL984 Instruction Bar** to the toolbar.

The customized toolbar is saved with the project.

Please also refer to the general toolbar features *(see Unity Pro, Operating Modes)*.

LL984 Instruction Bar

The **LL984 Instruction Bar** provides the LL984 function blocks.



Use the **LL984 Instruction Bar** button in the toolbar to show/hide the instruction bar.

To display the function blocks of the **EMTH** and **PCFL** families, click the respective button with an arrow in the lower right corner.

Clicking such a button opens a sub-toolbar containing the function blocks of the family.

You can drag and dock the instruction bar anywhere you like.

The position of the instruction bar is saved with the project, which means next time opening the project will restore the toolbar position.

Using Menus or Context Menu

You can activate the features provided by the toolbar also via the **Edit** menu or using the context menu of the LL984 Editor (right-click with the mouse).

General Restrictions

Introduction

For Unity Pro with LL984 Editor compared to legacy LL984 applications the restrictions described below apply.

Memory Addresses

The following memory address are used in Unity Pro:

Unity Pro	Legacy LL984 Application
%M (memory bit)	0x
%I (discrete input)	1x
%IW (input word)	3x
%MW (memory word)	4x

No Debugging Support

The Unity Pro LL984 Editor does not provide debugging support. Only the IEC languages (FBD, LD, SFC, ST, IL) provide debugging features.

Segment Scheduling

Segment scheduling is managed by the order of sections programmed in the MAST task.

- Unity Pro with LL984 Editor does not support executing 1 segment several times during a PLC scan, which was possible with legacy LL984 applications.
- The `Control` contact state can only be an `On` state to solve. Legacy LL984 applications allowed an option of `On` or `Off` to solve the segment.
- Unity Pro does not support the `Reset WDT` setting.
- Unity Pro does not support inverted condition variables neither for segments nor for networks.

If an I/O drop is associated with a logic segment without any logic programmed the segment is not created and therefore the associated I/O is not solved. Drops without logic must be explicitly set to scan associated I/Os.

Review the segment scheduler and logic segmentation and after the conversion add sections to the MAST task with the proper Remote I/O drop added that needs to be solved for the application.

For Modicon M340 target applications I/O scheduling is not possible from the segment. The MAST task updates the I/O modules.

Segments not in the right order after import of a ProWORX application

In case there is one or more segments, which are not marked for execution in the LL984 segment scheduler, the import will place the first of these segments as first LL984 segment in **Mast Task** segment tree.

The segments/sections are scanned in the order displayed in the **Project Browser**.

As this wrong placed segment has a condition `LL984_Segment_NotSolved`, this segment will not be scanned.

If the segment condition is removed, the segment must also be placed in the correct place in the **Mast Task** segment tree, to get correct execution order.

Segments/networks programmed but not listed in the segment scheduler are not imported.

All Segments which are disabled in LL984 segment scheduler are not scanned in Unity Pro.

Such segments are connected to a common variable, initialized at 0 named `LL984_Segment_NotSolved`.

To enable one or several segments, condition variable in the segment properties window must be deleted.

Import log file has an incorrect message - Negative segment conditions are not supported....

A wrong message may appear in the import log file after an import of a Concept **.asc* file: `Negative segment conditions are not supported. Segment condition must be adapted in Segment xx.`

This happens in case where a previously conditionally executed segment, was set to continuously before the Concept project was exported into a **.asc* file.

As the import of the continuously state is correct, there is no more action necessary.

Interrupt Handling

Unity Pro does not support interrupt handling with LL984 subroutines.

That means applications with the `ITMR` function and those that use the HLI 340 interrupt module with the `IMOD` instruction will need to be modified so that they make use of standard Unity Pro interrupt handling with I/O events.

If the original LL984 application contains an `IMOD` FFB programmed to service interrupts for the HLI input module, the LL984 subroutine sections will be imported into the Unity Pro application in the SR section of the MAST task. But they will not be operational with interrupt handling. In Unity Pro, the HLI I/O points that are configured as interrupts are linked to I/O events in the Events section. The LL984 subroutine program will have to be programmed (in one of the IEC languages) in the appropriate I/O EVT section.

Once completed, the SR sections that were imported into the MAST task need to be deleted.

Please also refer to Calling Subroutines *(see page 123)*.

Online Changes

Online changes in Unity Pro is not unlimited.

A message will be generated, if a certain percentage (e.g. 75%) of the available **Build → Build Changes** memory is already used.

The LL984 Editor of Unity Pro does not allow "direct" changes of code like in legacy LL984 systems.

A change performed online in an LL984 network will lead to a modification of the network and needs to be confirmed and downloaded by **Build → Build Changes**.

Forcing

In legacy LL984 applications forced bits remain in their state after a cold start.

In Unity Pro forced bits are reset on cold start.

Non-latched/Latched Coil Restrictions

For non-latched/latched coil restrictions please refer to Selecting Coils *(see page 94)*.

State Flow / Power Flow

Legacy LL984 applications provide switching between **State Flow** and **Power Flow** view.

Unity Pro only provides 1 view (**Power Flow**). Therefore no switching to **State Flow** is available.

Extended Memory

Unity Pro Quantum PLCs do not support SCADA packages that use Quantum extended memory reads and writes (Modbus FC0x14 and FC0x15).

Extended memory data from a legacy Quantum LL984 application will be moved to a Unity Pro defined variable location.

MMF (Modicon Motion Framework)

Unity Pro does not support the MMF (library of motion function blocks, loadable).

Modsoft SFC

Modsoft SFC and Modsoft macros will neither be converted to nor supported by Unity Pro.

Not Supported Legacy LL984 Instructions

For legacy LL984 instructions that are not supported by Unity Pro please refer to *Not Implemented Function Blocks (see Unity Pro, UnityLL984, Block Library)*.

If the imported LL984 Ladder Logic application contains unsupported LL984 logic instructions or user loadables, proceed as described under *Not Supported Legacy LL984 Instructions and User Loadables (see Unity Pro, Concept Application Converter, User Manual)*.

Function Blocks

L9_IMIO, No error code 16#2003 in case of IOFUN=3

When using the immediate I/O function block (L9_IMIO) with parameter IOFUN=3 (combined input and output operation) with a none-bidirectional I/O module there is no error-code 16#2003 reported.

EMTH FFBs: Results and error-code different between legacy LL984 and Unity Pro LL984

When using EMTH FFBs (L9E_...) with very small floating point values or with value=NAN/INF the result of the operation or the error code retrieved by the EMTH L9E_ERLOG FFB may differ between legacy LL984 and Unity Pro LL984.

Examples:

- If the input value of L9E_SINE is 1.363573E-36, the L9E_ERLOG FFB in ProWORX32 reports the error code 1A01 (FP Underflow), while in Unity Pro LL984 there is no error code reported. In Unity Pro LL984 the Underflow error is reported if the input value is ≤ 1.175E-38.
- If you compare the FP values 9.18E-41 and 0.0 using the L9E_CMPFP function block, the result in legacy LL984 is Equal and in Unity Pro LL984 the result is Greater.
- If the input value of the L9E_SQRFP (square root) FFB is +NAN, the error code retrieved by the EMTH L9E_ERLOG FFB in Unity Pro is 16#1704 (FP value invalid), while in legacy LL984 the error code is 16#1702 (FP Overflow).

PCFL FFBs: Different results between legacy LL984 and Unity Pro LL984

The calculation of the formula (A*B*C)/D in the L9P_CALC FFB delivers different results in legacy LL984 and in Unity Pro LL984 in case of A=1.0 and B=C=D=9.18E-41. In Unity Pro LL984 the result is 0.0 and in legacy LL984 the result is 9.18E-41.

XMIT, Not present in Library error. Opcode must be changed to 1F

L9_XMIT now is offered as an EFB always available, while legacy LL984 projects know it as a loadable. Opcode is no more needed and must be set to 1F before importing application to Unity Pro.

L9_ID and L9_IE function blocks do not disable/ enable the ITCNTRL when set in Mode 3

With the L9_ID/L9_IE function block you can mask/unmask interrupts. The parameter INT_TYPE specifies the type of operation:

- INT_TYPE=1
  A global interrupt masking is performed, that means all I/O and timer interrupts are masked (L9_ID) or unmasked (L9_IE). Masking of interrupts can be done for a series of PLC scans. The masking starts when the L9_ID block is executed and it ends when the L9_IE block is executed. The L9_ID/L9_IE block with INT_TYPE=1 has the same effect as the IEC function block MASKEVT/UNMASKEVT.
- INT_TYPE=2
  Not supported in Unity Pro LL984.

- `INT_TYPE=3`
  Only timer interrupts are masked/unmasked. This mode can be used to protect a short sequence of code in your application from being interrupted by a timer event. You cannot disable timer interrupts for several PLC scans. The `L9_ID`/`L9_IE` function block with `INT_TYPE=3` has the same effect as the IEC function block `ITCNTRL` with parameter `ENABLE=False` (`L9_ID`) or `Enable=True` (`L9_IE`) and `EVENT=1..32`.

State RAM Viewer

LL984 is always referencing the symbol and not the direct address so when you assign a contact to the Variable %MW651 unity Pro create automatically the symbol _400651.

Unity pro will do its reference on this symbol and not on the location of the State RAM so it is this symbol which is seen as used by the language and not the %MW651.

Nevertheless the variable is seen with the filter on **variable** as used by the application.

Other mechanisms can be used to check if the variables are used by the application as search, cross references and Used column in the data editor.

Chapter 3
Accessing Tasks and Sections

Overview

Accessing Sections

This chapter describes the specifics added in **Project Browser** in order to support LL984 segments and networks.

For standard features please refer to *Accessing Sections (see Unity Pro, Operating Modes)*.

Accessing Tasks

For information on how to create and modify program tasks please refer to *Access to Tasks (see Unity Pro, Operating Modes)*.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Creating LL984 Segments and Networks	34
Modifying LL984 Segment Properties	38
Modifying LL984 Network Properties	40
Scheduling LL984 Segments and Network	42
Deleting, Cutting, Copying and Pasting LL984 Segments and Network	43

Creating LL984 Segments and Networks

Introduction

The logic of LL984 is programmed in networks.

Several networks are grouped in segments.

Each network contains a matrix of 7 rows and 11 columns, where you can place contacts, coils, function blocks etc.

At the beginning of each segment's code execution the inputs of a RIO drop can be read and at the end the outputs of the same or another RIO drop can be written.

Recommendations

It is recommended not to use more than 64 segments and 5,000 networks in an application.

Restrictions

For creating LL984 segments and networks the following rules apply:

- LL984 segments can only be created when the **Ladder Logic 984 (LL984)** project setting is activated via **Tools → Project Settings → Program → Languages**.
- LL984 segments can only be created in the **MAST** task of the application, they are not allowed in the **FAST** and **AUX** tasks.
- Only one LL984 segment can be created inside the subroutine (**SR Sections**) node of the **MAST** task.
- LL984 segments can not be created under **Timer Events** or **I/O Events** nodes.
- LL984 segments can not be created in DFBs.
- For action and transition sections in SFC, LL984 is not allowed as language.
  Only ST, IL, FBD and LD are allowed as language.
- LL984 segments and sections with Unity Pro IEC languages can be mixed inside the **Sections** node of the **MAST** task. The same is valid inside the subroutine (**SR Sections**) node of the **MAST** task.
- LL984 networks can only be created as children of LL984 segments.
- Only LL984 networks are allowed as children of LL984 segments, no sections with IEC languages can be created there.

Creating a Segment

Step	Action
1	Left-click the **Sections** directory of the **MAST** task, or place the cursor using the arrow keys on this directory.
2	Click the right mouse button or use **Shift+F10** to select **New Section** in the context menu. **Result:** The properties dialog for the segment is opened.
3	Enter the section (segment) name. The section (segment) name must be unique in the entire project and conform to the general naming conventions of Unity Pro.
4	Select **LL984 Segment** as programming **Language** of the segment.
5	Select the functional module to which the segment is attached from the **Localization** tab if necessary. Also refer to *Functional modules (see Unity Pro, Operating Modes)*.
6	Define a boolean variable in the **Condition** tab with which the segment can be activated/deactivated (segment active when variable=1) if required. You have the following options to select variables: ● Enter the name of the variable/address directly or copy it from the clipboard. ● Use the **...** button to open a **Data Selection** dialog.
7	**Only applies to Quantum projects:** Assign a read RIO drop and a write RIO drop to the segment in the **Remote IO** tab if required.
8	Enter a comment in the **Comment** tab if desired (max. 1024 characters).
9	If you wish to create additional segments, confirm the entries with **Apply**. In this case, the dialog will remain open and you can generate additional segments. If you do not wish to create additional segments, confirm the entries with **OK**. In this case, the dialog will close and the generated segment is displayed as the last element in the **Sections** directory. **NOTE:** The position of sections and segments inside the **Structural view** matches exactly the execution sequence and can be changed via drag-and-drop.

Creating a Network

Step	Action
1	Left-click an **LL984 Segment** in the **Sections** directory of the **MAST** task, or place the cursor using the arrow keys on this directory.
2	Click the right mouse button or use **Shift+F10** to select **New 984 Network** in the context menu. **Result:** The properties dialog for the network is opened.
3	Enter the network name. The network name must be unique in the entire project and conform to the general naming conventions of Unity Pro.
4	**LL984 Network** is preselected for the **Language** of the network and can not be changed.
5	Define a boolean variable in the **Condition** tab with which the segment can be activated/deactivated (segment active when variable=1) if required. You have the following options to select variables: ● Enter the name of the variable/address directly or copy it from the clipboard. ● Use the command button **...**to open a **Data Selection** dialog.
6	Enter a comment in the **Comment** tab if desired.
7	Confirm the entries with **OK**. The dialog will close and the generated network is displayed as the last element in the **LL984 Segment** directory. **NOTE:** The position of networks inside the **Structural view** matches exactly the execution sequence and can be changed via drag-and-drop.

A new network is normally inserted as the last element in the **LL984 Segment** directory.

To insert a new network directly before a selected network use **Insert Network …** from the context menu.

Creating a Network for Subroutines

Step	Action
1	Left-click the **SR Sections** directory of the **MAST** task, or place the cursor using the arrow keys on this directory.
2	Click the right mouse button or use **Shift+F10** to select **New SR Section** in the context menu. **Result:** The properties dialog for the SR section (**LL984 Segment**) is opened.
3	Enter the SR section (LL984 segment) name. The SR section (**LL984 Segment**) name must be unique in the entire project and conform to the general naming conventions of Unity Pro.

Step	Action
4	Select **LL984 Segment** as programming **Language** of the segment. **NOTE:** There can be only 1 LL984 SR segment in a project. So when there is already 1 existing you can no longer select **LL984 Segment** as programming **Language** for a new SR section.
5	Enter a comment in the **Comment** tab if desired (max. 1024 characters).
6	Confirm the entries with **OK**. The dialog will close and the generated LL984 segment is displayed in the **SR Section** directory.
7	Left-click the **LL984 Segment** in the **SR Sections** directory of the **MAST** task, or place the cursor using the arrow keys on this directory.
8	Click the right mouse button or use **Shift**+**F10** to select **New 984 Network** in the context menu. **Result:** The properties dialog for the network is opened.
9	Enter the network name. The network name must be unique in the entire project and conform to the general naming conventions of Unity Pro.
10	**LL984 Network** is preselected for the **Language** of the network and can not be changed.
11	Enter a comment in the **Comment** tab if desired.
12	Confirm the entries with **OK**. Tthe dialog will close and the generated network is displayed as the last element in the **LL984 Segment** directory. **NOTE:** The position of networks inside the **Structural view** matches exactly the execution sequence and can be changed via drag-and-drop.

Displaying or Modifying Segment/Network Properties

Step	Action
1	Left-click on the desired segment/network, or place the cursor on this section/network with the arrow keys.
2	Click the right mouse button or use **Shift**+**F10** to select **Properties** in the context menu. **Result:** The **Properties** dialog for is opened.
3	Implement the desired changes.
4	Confirm all entries with **OK**.

Exporting/Importing Segments/Networks

Please refer to Exporting/Importing LL984 Segments/Networks *(see Unity Pro, Concept Application Converter, User Manual)*.

Modifying LL984 Segment Properties

Introduction

This section describes the specifics of the LL984 Editor referring to modifying LL984 segment properties.

For standard features please refer to *Accessing Sections (see Unity Pro, Operating Modes)*.

Protection

The **Protection** property is not available at the moment.

Localization

You can only attach LL984 segments to a functional module and **not** LL984 networks. The networks of a segment are implicitly attached to the functional module of their segment.

You can attach an LL984 segment to a functional module either in the **Localization** tab of its properties dialog or in the **Project Browser** via drag-and-drop from the **Structural View** to the **Functional View**.

You can detach an LL984 segment from a functional module either in the **Localization** tab of its properties dialog or by selecting **Detach** from the segment´s context menu.

Condition

You can modify the **Condition** of an LL984 segment in the **Condition** tab of its properties dialog.

When you configure a boolean variable in this tab, the state of this variable is checked at the beginning of the LL984 segment's code execution.

When the variable´s state is 0, the segment and all its networks are not executed.

When the variable´s state is 1 or when no condition variable is defined for the segment, it depends on the condition variables of the networks whether the networks are executed or not.

Remote I/O

You can synchronize the Remote I/Os of an LL984 segment in the **Remote I/O** tab of its properties dialog.

Here you can configure which Remote Quantum drop is read before and which one is written after execution of the LL984 segment.

Drops which are not yet configured are displayed in red inside the list box.

Not configured drops can be selected here, but when they are not configured before next **Build**, the build will not be successful and a diagnostic message will be displayed.

NOTE: This property is only available for Quantum projects.

Comment

You can enter a comment to an LL984 segment in the **Comment** tab of its properties dialog.

The maximum size of a comment is 1024 characters.

The LL984 Subroutine Segment

You can only have 1 LL984 segment under the **SR sections** node of the **MAST** task.

The properties dialog of this LL984 subroutine segment only provides the **General** and the **Comments** tabs.

The **Is called** check book in the **General** tab is greyed and not checked.

This is done because in LL984, subroutines can be called via a memory word and so you only know at run time whether a subroutine is called or not.

You can not define a condition variable for the LL984 subroutine segment.

Modifying LL984 Network Properties

Introduction

This section describes the specifics of the LL984 Editor referring to modifying LL984 network properties.

For standard features please refer to *Accessing Sections (see Unity Pro, Operating Modes)*.

Protection

The **Protection** property is not available at the moment.

Localization

You can only attach an LL984 segment to a functional modules. The networks of a segment are implicitly attached to the functional module of their segment.

Condition

You can modify the **Condition** of an LL984 network in the **Condition** tab of its properties dialog.

When you configure a boolean variable in this tab, the state of this variable is checked at the beginning of the LL984 network's code execution.

When the variable´s state is 0, the variable and its networks are not executed.

When the variable´s state is 1 or when no condition variable is defined for the network, it depends on the condition variables of the networks whether the networks are executed or not.

You can define a condition variable for each network.

NOTE: Condition variables do not have influence on the execution of an `L9_SKP` block, that means when you have a skip over 4 networks, you skip the remainder of the current network and the next 3 networks independent of their condition variables.

Remote I/O

You can synchronize the Remote I/Os only for LL984 segments and not for LL984 networks.

Comment

You can enter a comment to an LL984 network in the **Comment** tab of its properties dialog.

The maximum size of a comment is 1024 characters.

User Directory/Hyperlink

For LL984 networks you can add hyperlinks by selecting **Add Hyperlink** from the network´s context menu.

For details please refer to chapter *Hyperlinks (see Unity Pro, Operating Modes)*.

You can not add user directories to LL984 networks.

LL984 Networks in Subroutine Segments

The properties dialog of an LL984 network in a subroutine (SR) segment only provides the **General** and the **Comments** tabs.

The **Is called** check box in the **General** tab is greyed and not checked, because in LL984 subroutines can be called via a memory word and so you only know at run time whether a subroutine is called or not.

You can not define a condition variable for an LL984 network in a subroutine (SR) segment.

Scheduling LL984 Segments and Network

Introduction

Sections and LL984 segments (and also LL984 networks inside segments) are executed in the sequence in which they are displayed in the **Structural View** of the **Project Browser**.

To change this sequence, you can drag-and-drop sections, LL984 segments or LL984 networks inside the **Structural View**.

Restrictions

For dragging-and-dropping LL984 segments and networks the following rules apply:

- Sections (ST, LD, …) can not be dropped into LL984 segments.
- LL984 segments can not be dropped into other LL984 segments.
- LL984 networks can only be dropped into LL984 segments.
  LL984 networks can be dropped into the LL984 subroutine segment under the **SR Sections**) node of the **MAST** task.
  After dropping the networks, they are re-analyzed latest during next **Build**, because e.g. `L9_LAB` function blocks are only allowed in networks of the LL984 segment in the **SR Sections** node.
- When an LL984 network has an activation condition and is dragged-and-dropped into the LL984 subroutine segment, the LL984 network loses its activation condition because subroutine networks do not have activation conditions.

Drag-and Drop Between Applications

You can not drag-and drop LL984 segments and networks from one Unity Pro application to another.

To copy sections between applications you have to use the import/export feature of Unity Pro.

Deleting, Cutting, Copying and Pasting LL984 Segments and Network

Deleting LL984 Segments and Networks

You can delete LL984 segments and networks in the **Structural View** of the **Project Browser**.

To delete an LL984 segment or network either select **Delete** from its context menu or press the **DEL** key.

If you want to delete an LL984 segment, a message is displayed that all its networks will be deleted, too.

You have to confirm this message to delete the LL984 segment.

Cutting, Copying and Pasting

You can not cut, copy and paste LL984 segments and networks in a Unity Pro application.

Alternatively you can use the import/export feature on segment or network level.

Chapter 4
Unity Pro Software Options

Overview

This chapter describes the specifics of the LL984 Editor referring to Unity Pro software options.

For standard features please refer to *Unity Pro Software Options (see Unity Pro, Operating Modes)*.

What Is in This Chapter?

This chapter contains the following sections:

Section	Topic	Page
4.1	Project Settings	46
4.2	Options	51

Section 4.1
Project Settings

Program

Overview

This chapter describes the specifics of the LL984 Editor referring to Unity Pro project settings (**Tools → Project Settings**).

For standard features please refer to *Project Settings (see Unity Pro, Operating Modes)*.

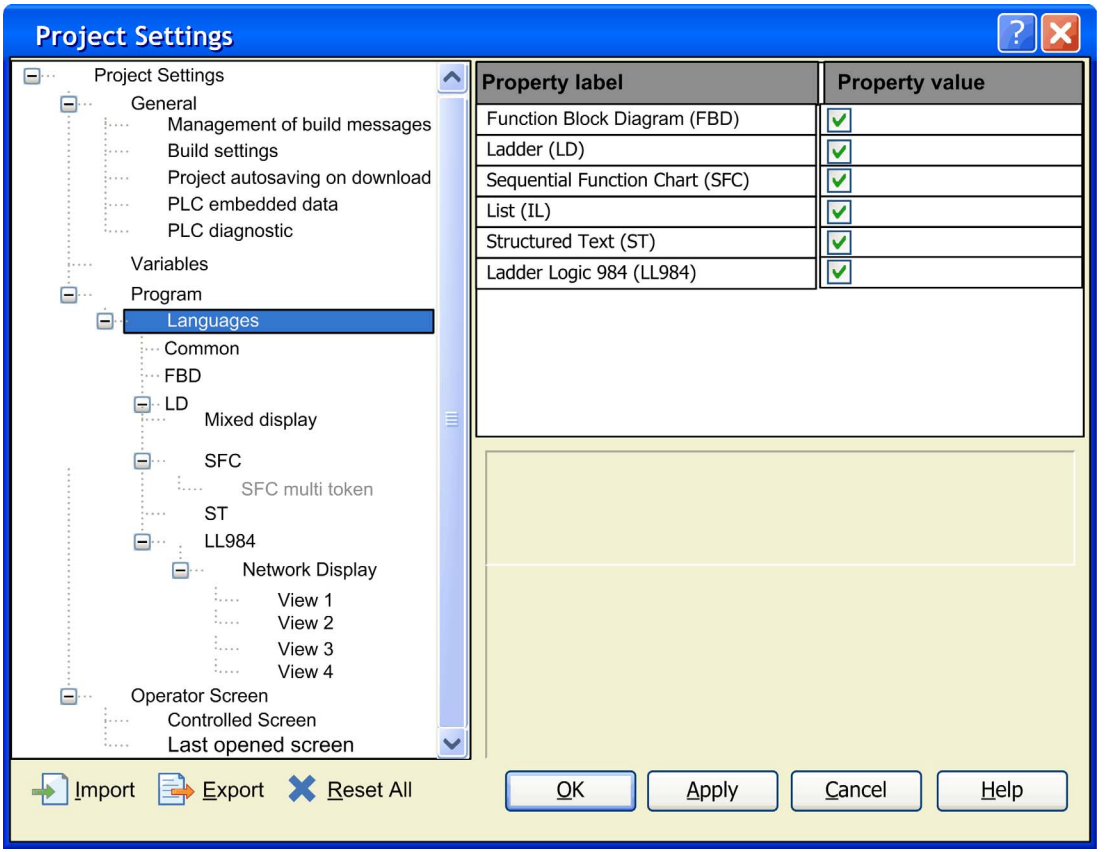
Introduction

This project´s configuration settings can be accessed via **Tools → Project Settings → Program**. It contains FBD, LD, SFC, ST and LL984 settings.

As with other Unity Pro project settings, the LL984 specific project settings are also stored as part of the application.

Representation

Appearance of the **Languages** settings



Items of the Languages Settings

Item	Description
Function Block Diagram (FBD)	When these check boxes are checked, the respective languages can be used in your project.
Ladder (LD)	
Sequential Function Chart (SFC)	
List (IL)	
Structured Text (ST)	
Ladder Logic (LL984)	When this check box is checked, the LL984 language can be used in your project.

NOTE:
For modifying the **Ladder Logic (LL984)** setting the following restrictions apply:
- This setting is disabled for applications using Quantum Safety PLCs, as LL984 segments and networks can only be created for non-safety applications.
- To modify this setting you have to go offline.

Item of the LL984 Setting

This LL984 setting is enabled only, if the LL984 library setup was installed.

Item	Description
SKP enabled	When this check box is checked, the L9_SKP function block can be used in your project.

NOTE: Modifying this setting causes a re-analysis of the LL984 networks. This is also possible in the online mode.

Items of the Network Display Settings

Item	Description
Printing View	Using this list you select **view1** to **view4** defined under items **View 1** to **View 4** (see below). The default view is **view4**. The view you select for the **Printing View** setting is used for printing the LL984 networks.
Column Width	Using this field you can enter the column width as a floating point value (**1.0** to **3.0**). For example a value of 3.0 means a 3 times larger column width. The default width is **1.0**.

NOTE: Modifying the **Column Width** setting does not cause a new **Build**.

Items of the View 1 to View 4 Settings

The LL984 Editor can display various information concerning variables attached to coils, contacts and function block nodes.

According to the settings for **View 1** to **View 4** the LL984 Editor displays the following information:
- address (e.g., %M100)
- 984-address (e.g., 400100)
- variable name
- variable comment
- descriptors 1 to 9
- topological address

You can switch between the different views defined using **CTRL+W** or using the **Change View Mode** button.

Item	Description
Line 1 ... **Line 7**	For each line, you can select which information should be shown: ● **blank** ● **address** ● **984-address** ● **variable** ● **comment** ● **descriptor 1 to descriptor 9** ● **topological address**
Max characters per cell width	Defines the maximum number of characters visible in one cell (5 to 32). Increasing this value leads to a smaller font size and vice versa. The default is 10 characters. **NOTE:** A high values (for example 32) may lead to a very small (unreadable) font size, but such a value makes sense when used in conjunction with a larger **Column Width** setting (like 2.0 or 3.0).

Default settings for **View 1** to **View 4**

Setting	View 1	View 2	View 3	View 4
Line 1	blank	blank	blank	blank
Line 2	blank	blank	blank	blank
Line 3	blank	blank	blank	blank
Line 4	blank	blank	blank	blank
Line 5	blank	blank	blank	comment
Line 6	blank	variable	984-address	variable
Line 7	address	address	address	address
Max characters per cell width	10	10	10	10

NOTE:
For the **View 1** to **View 4** settings the following restrictions apply:

- If the value **variable** is selected in consecutive lines (for example **Line 1** and **Line 2**) the variable name is broken into n lines, if it is longer than the cell width.
  If the value **variable** is selected, for example, for **Line 1** and **Line 3** and if for **Line 2** any other value is selected, the variable name is truncated at the end of **Line 1** and is not broken into **Line 3**.
  Line 3 starts with the first character of the variable name again.

- If the value **comment** is selected in consecutive lines (for example **Line 1** - **Line 3**) the variable comment is broken into n lines, if it is longer than the cell width.
  If the value **comment** is selected, for example, for **Line 1** and **Line 3** and if for **Line 2** any other value is selected, the variable comment is truncated at the end of **Line 1** and is not broken into **Line 3**.
  Line 3 starts with the first character of the variable comment again.
- If all 7 lines are configured as **blank**, the LL984 Editor displays the variable name in **Line 7** automatically.
- **Max characters per cell width**
  The calculated font height is applied for all kinds of displayed information (address, variable, comment etc.).
- Blank lines at the top are not displayed. Blank lines in the middle or at the bottom are displayed as empty lines.

Section 4.2
Options

Data and Languages

Overview

This chapter describes the specifics of the LL984 Editor referring to Unity Pro options (**Tools → Options**).

For standard features please refer to *Options (see Unity Pro, Operating Modes)*.

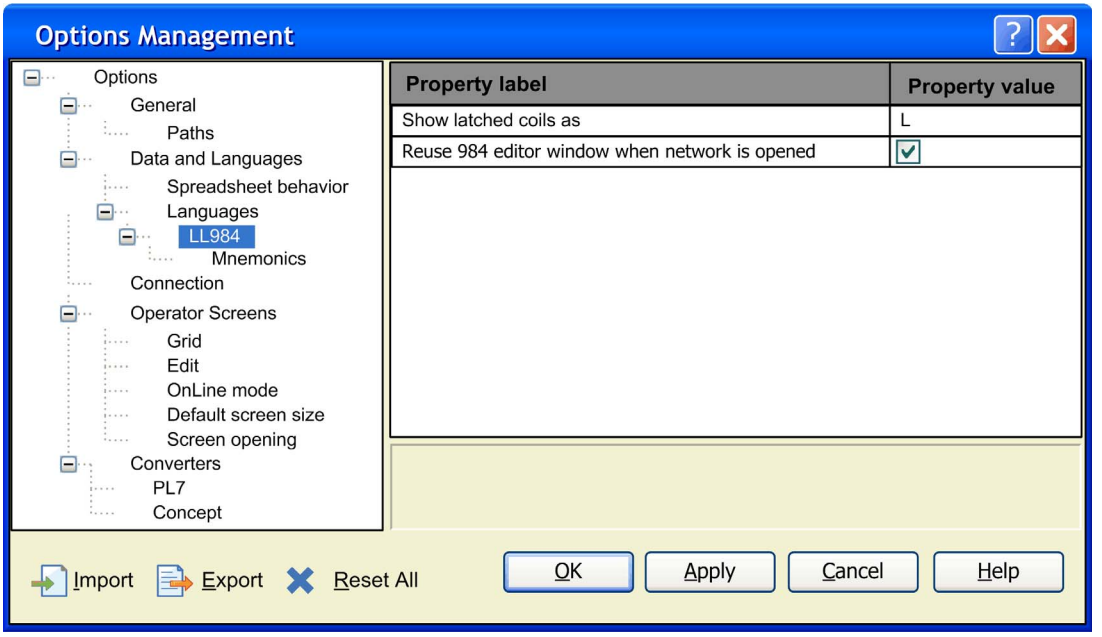
Introduction

This configuration section can be accessed via **Tools → Options → Data and Languages**. It contains settings for entering data.

For standard features please refer to *Data and Languages (see Unity Pro, Operating Modes)*.

Representation

LL984 options



Items of the LL984 Options

Item	Description
Show latched coil as	Using this list you can select which letter (L or M) will be displayed inside a latched coil. The default letter that will be displayed is an L.
Reuse 984 editor window when network is opened	If this check box is checked only 1 window is used for **Open** commands on LL984 networks. If this check box is not checked, for each **Open** command on an LL984 network, a new window is opened.

NOTE: For modifying the settings mentioned above, the following restrictions apply:

- Modifying the **Show latched coils as** setting will lead to a refresh of the editor window of the opened LL984 networks.
- Modifying the **Reuse 984 editor window when network is opened** setting will only take effect next time an LL984 editor is opened.

Item of the Mnemonics Options

Under **LL984 → Mnemonics** the **Property label** column lists the contacts and coils and an assortment of functions blocks available for LL984 networks.

In the **Property value** column you can enter a mnemonic for each listed element.

A mnemonic can consist of up to 4 alphanumeric signs.

Using Mnemonics

As you start typing text, while the current cell marker is on an empty cell in the LL9894 editor, a small edit box opens in the editor.

Entering a mnemonic (first key typed will be taken over into the edit box) and pressing the ENTER key will insert the corresponding LL984 element at the current cell.

If the cells needed to insert the element are already occupied, a message box will inform you and the element will not be inserted.

Chapter 5
Common Functions for Graphical Languages

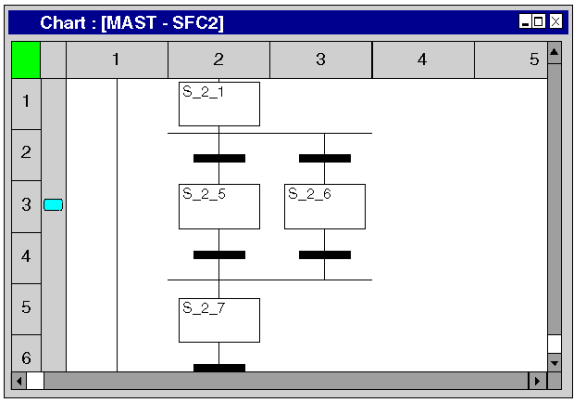
Using Bookmarks in Graphical Languages Sections

Introduction

Bookmarks are used to mark positions in the margin (gray column on the left edge of the section/network) and allow the marked positions to be easily found in different language sections/networks.

You can navigate through a list of bookmarks, displayed in the **Tools → Bookmarks** dialog.

An example of a bookmark in SFC:



The current bookmark is indicated using a horizontal arrow ( ).

Bookmark are displayed in different colors.

Created	Enabled/Disabled	Color
manually	enabled	blue
manually	disabled	light gray
automatically	enabled	gray
automatically	disabled	light gray

Setting Bookmarks

You have the following options to set a bookmark:

- Execute the command **Edit → Bookmarks > → Toggle Bookmark**. (There must not be any current bookmark in the selected area).
  The bookmark is inserted in the margin (gray column on the left edge of the section/network) and is marked as the current bookmark.
- Press the **Ctrl+F2** key combination. (There must not be any current bookmark in the selected area).
  The bookmark is inserted in the margin (gray column on the left edge of the section/network) and is marked as the current bookmark.
  or
- Place the mouse pointer in the margin (gray column on the left edge of the section/network) at the position to be marked and use **Toggle Bookmark** from the context menu for the margin (right mouse click).
  The bookmark is inserted at the mouse pointer and marked as the current bookmark.

Removing Single Bookmarks

You have the following options to remove a bookmark:

- Select the bookmark to be deleted as the current bookmark (e.g. with **Next Bookmark**) and use the **Edit → Bookmark > → Toggle Bookmark** command.
- Select the bookmark to be deleted as the current bookmark (e.g. with **Next Bookmark**) and press the **Ctrl+F2** key combination.
  or
- Place the mouse pointer on the bookmark to be removed and use **Toggle Bookmark** from the context menu of the bookmark (right mouse click).
- You can also remove a bookmark using the context menu of the **Bookmarks** dialog.

Removing All bookmarks

You have the following options to remove all bookmarks in the current section/network:

- Use the **Edit → Bookmark> → Clear all Bookmarks** menu command.
  or
- Press the **Ctrl+Shift+F2** key combination.
- You can also remove all bookmarks using the context menu of the **Bookmarks** dialog.

Searching for Bookmarks (Search Down)

You have the following options to search for bookmarks (search down):

- Use the **Edit → Bookmark> → Next Bookmark** menu command.
- Use the **Edit → Go to → Bookmark → Next** menu command.
  or
- Press the **F2** key.

If a current bookmark is selected, a jump is made to the next one and it is marked as the current bookmark.

The bookmarks are traversed according to their order of appearance in the **Bookmarks** dialog.

Searching for Bookmarks (Search Up)

You have the following options to search for bookmarks (search up):

- Use the **Edit → Bookmark> → Previous Bookmark** menu command.
- Use the **Edit → Go to → Bookmark → Previous** menu command.
  or
- Press the **Shift+F2** key combination.

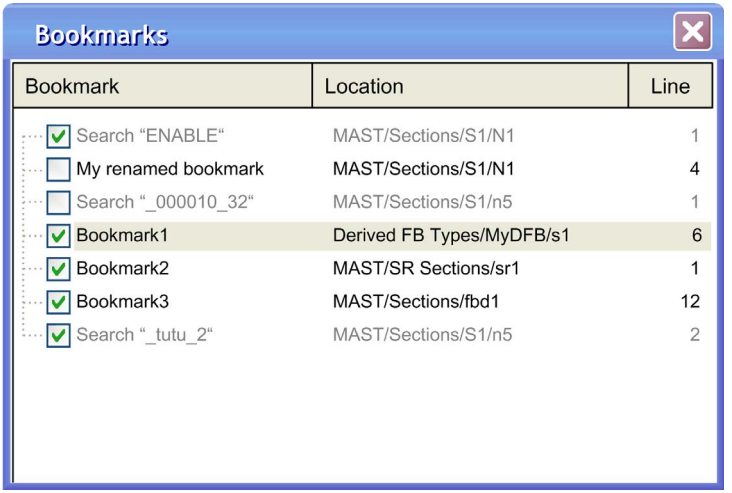
If a current bookmark is selected, a jump is made to the previous one and it is marked as the current bookmark.

The bookmarks are traversed according to their order of appearance in the **Bookmarks** dialog.

Bookmarks Dialog

To open the bookmarks dialog use **Tools → Bookmarks**.

A list of the project´s bookmarks is displayed in the **Bookmarks** dialog.



Element	Description
Bookmark column	List of the project´s bookmarks. Automatically created bookmarks (see below) are displayed in gray and can not be renamed.
Location column	Location of the bookmark (section, network or DFB) referring to the **Project Browser**.
Line column	Line in the section/network where the bookmark is located.
check boxes in the **Bookmark** column	Activate this check box to be able to jump to this bookmark by means of **Next Bookmark**/**Previous Bookmark**. By default for newly created bookmarks the check box is activated.
Delete/**Delete all**	Right click in the dialog and select from the context menu: • **Delete**: to delete the selected bookmark • **Delete all**: to delete all bookmarks in the project

Creating bookmarks is done in the language editors.

Deleting and renaming bookmarks can be done in the **Bookmarks** dialog, too.

NOTE: Renaming of bookmarks can only be done with created bookmarks. Automatically created bookmarks can not be renamed.

Bookmark Commands in Language Editors

The bookmark commands in language editors affect the **Bookmarks** dialog as follows:

- **Toggle Bookmark** creates a bookmark, visible also in the **Bookmarks** dialog. It will be appended to the list of bookmarks.
- **Next Bookmark** jumps to the next bookmark in the list of bookmarks
- **Previous Bookmark** jumps to the previous bookmark in the list of bookmarks.
- **Delete Bookmark** deletes the selected bookmark.
- **Delete all Bookmarks** deletes only the bookmarks inside the current section/network.

NOTE: The **Bookmark** tab of the **Edit → Go To** dialog only displays the bookmarks of the current section.

Naming Bookmarks

Created bookmarks are named as **Bookmark1**, **Bookmark2** and so on.

In the **Bookmarks** dialog you can rename the bookmarks.

Automatically Created Bookmarks

Automatically created bookmarks are named as **Search"<Search String>"**.

Automatically created bookmarks are displayed in gray and can not be renamed.

The maximum number of automatically created bookmarks is 30.

Automatic creation of another bookmark will insert the new bookmark as the first one in the bookmark list and will delete the bookmark at the end.

For automatically created bookmarks please refer to Tracing Variables / Addresses *(see page 143)*.

Navigating to Bookmarks

In the **Bookmarks** dialog double-click a line in the list of bookmarks to open the respective section/network with the selected bookmark.

If the check box in front of a bookmark line is not activated, you can not jump to this bookmark by means of **Next Bookmark**/**Previous Bookmark**.

Saving Bookmarks

Bookmarks are saved with your application.

Deleting Bookmarked Lines

If you delete a section/network or a bookmarked line in a section/network, the corresponding bookmarks are deleted, too.

Undo will not restore the bookmarks.

Chapter 6
LL984 Editor

Overview

This chapter describes the specifics of the LL984 Editor.

What Is in This Chapter?

This chapter contains the following sections:

Section	Topic	Page
6.1	Creating a Program with LL984 Programming Language	60
6.2	Editing Contacts	83
6.3	Editing Coils	93
6.4	Editing Function Blocks	101
6.5	Editing Links	106
6.6	Editing Equation Network Blocks	112
6.7	Assigning Variables to LL984 Objects	119
6.8	Calling Subroutines	123
6.9	Entering Comments	125

Section 6.1
Creating a Program with LL984 Programming Language

Overview

This section describes the basics for creating a program in the LL984 programming language.

What Is in This Section?

This section contains the following topics:

Topic	Page
Project Settings for LL984 Programs	61
Structure of an LL984 Program	62
Creating an LL984 Program	65
Syntax and Semantics Check During Programming	68
Network Display	71
Navigating with the Keyboard	72
Selecting Objects	74
Deleting, Cutting, Copying, Pasting and Moving Objects	76
Displaying the Properties	80
Inserting and Deleting Rows/Columns	81

Project Settings for LL984 Programs

Introduction

Before creating an LL984 program, you should activate the following project settings:

- **Ladder Logic 984 (LL984)**
  (**Project Settings → Program → Languages**)
  To be able to create LL984 sections and networks.
- **Allow dynamic arrays (ANY_ARRAY_XXX)**
  (**Project Settings → Variables**)
  To be able to use dynamic arrays with the LL984 function blocks.

Structure of an LL984 Program

Introduction

The structure of an LL984 program corresponds to a rung for relay switching.

The left power rail is located on the left side of the LL984 Editor. This left power rail corresponds to the phase (L conductor) of a rung.

The right power rail corresponds to the neutral conductor.

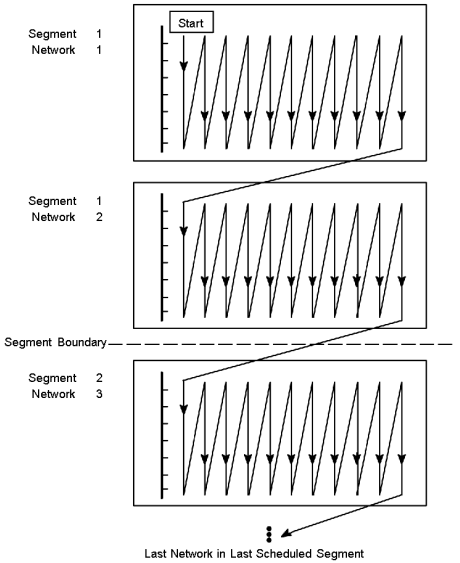
A group of connected objects that are not connected to other objects (except for the power rails), is called a rung.

Properties of an LL984 program

In LL984 program:

- The logic of LL984 is programmed in networks.
- Several networks are grouped in segments.
- Each network contains a matrix of 7 rows and 11 columns, where you can place contacts, coils, function blocks etc.
- LL984 programs are cell oriented, i.e. only 1 object can be placed in each cell.
  With the exception of vertical connections. They may coexist with other objects (contact, coils, function blocks).

- LL984 segments (and also LL984 networks inside segments) are executed in the sequence in which they are displayed in the **Structural View** of the **Project Browser**. To change this sequence, you can drag-and-drop LL984 segments or LL984 networks inside the **Structural View**.
  Objects in a network are executed top to bottom, left to right.



- A syntax and semantics check is performed immediately after the statement is entered. The result of this check is displayed in colored text and objects.
  Please refer to Syntax and Semantics Check During Programming *(see page 68)*.
- Syntactically or semantically incorrect segments or networks can be saved.

LL984 Operating Modes

NOTE: to detect the first cycle in RUN, the recommended solution is to use **%S21** system bit.

LL984 Program Objects

The objects of the LL984 programming language help to divide a network into a number of:

- contacts *(see page 83)*
- coils *(see page 93)*
- function blocks
- equation network blocks *(see page 112)*
- jumps and subroutine calls *(see page 123)*

These objects can be linked with each other through:

- boolean connections and links *(see page 106)*
- variables

Comments for the logic of the program can be added using text objects *(see page 125)*.

Edit and View Functions

- Selecting objects
- Deleting objects
- Cutting, copying and pasting objects
- Moving objects (also between different LL984 sections)
- Replacing objects
- Undo and Redo
- Using Bookmarks *(see page 53)*
- Searching and replacing variables and function blocks *(see page 129)*

FFB Input Features

The LL984 Editor provides 2 additional features to add function blocks to a network:

- LL984 Instruction Bar *(see page 25)*
- mnemonics *(see page 52)*

Using mnemonics *(see page 105)* you can also add coils and contacts to a network.

Online Functions

Displaying and modifying variable values using:

- animation tables *(see page 148)*
- instruction editors *(see page 154)*

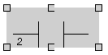
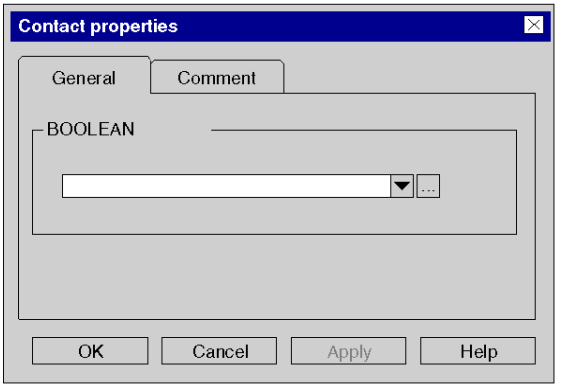
Creating an LL984 Program

Introduction

The LL984 Editor window is made up of cells and 1 object can be placed in each one. The cells are separated visually using a grid which can be switched off (**View → Grid**).

Creating an LL984 Program with the Mouse

Process the following steps to create an LL984 program with the mouse:

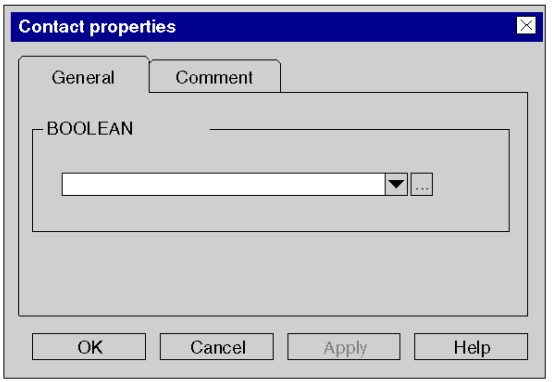
Step	Action
1	Create an LL984 network. *(see page 34)*
2	Select the desired object using: ● the menu commands in the **Edit → New → <Object>** menu or ● the symbols for the objects in the toolbar **Result:** The mouse pointer indicates the selected object (placement mode).
3	Click the target cells in the LL984 section. **Result:** The selected object is inserted and the selection mode is active again. Example:  Return to selection mode by clicking the  symbol or pressing the **ESC** key.
4	To enter the respective actual parameters (variable/address), double-click the object. **Result:** The properties dialog box for the object is opened. Example: 

Step	Action
5	Enter the desired actual parameter and a comment (if required) and confirm the entries with **OK**. **Result:** The names of the associated variables are displayed above the object. **NOTE:** Entering a binary address opens the **Create variable** dialog. By this you can create a variable associated with the entered address.
6	Repeat these steps until all objects are entered.

Creating an LL984 Program with the Keyboard

Process the following steps to create an LL984 program with the keyboard:

Step	Action
1	Create an LL984 network. *(see page 34)*
2	Using the keyboard, place the field with the gray background on the cell where the object should be inserted.
3	Select the desired object using the function keys for the objects. **Result:** The cursor symbol indicates the selected object.
4	Confirm the selection using the **ENTER** key. **Result:** The selected object is inserted into the cell with the gray background and the gray field is automatically shifted to the next cell. Example:  To return to selection mode, press the **ESC** key.

Step	Action
5	To enter the respective actual parameters (variable/address), use the **Properties** menu command from the shortcut menu. **Result:** The properties dialog box for the object is opened. Example: 
6	Enter the desired actual parameter and a comment (if required) and confirm the entries with **OK**. **Result:** The names of the associated variables are displayed above the object. **NOTE:** Entering a binary address opens the **Create variable** dialog. By this you can create a variable associated with the entered address.
7	Repeat these steps until all objects are entered.

Syntax and Semantics Check During Programming

Introduction

Syntax and semantics check is performed directly when creating the program.

The result of this check is displayed in 3 formats:

- Directly in the program section with colored objects or texts.
- As tooltip if the cursor is placed on an incorrect object.
- In the output window, if **Build → Analyze** is selected.

NOTE: Warnings are not visualized in the program section.

Colors of Objects/Texts

Color	Description
black	Syntax and semantics are correct.
blue	Possible causes: ● Associated variable is not declared. ● Data type of the variable does not match the data type of the pin. ● Input or output pin not connected.
red wavy line below text	invalid text. For example, variables which are not declared, variables with an incorrect data type, etc.

Tooltips

If the cursor is placed over an incorrect object, a tooltip is displayed with a brief description of the cause of the error/warning.

Messages in the Output Window

Error and warning messages are displayed in the output window after **Build → Analyze**.

Double-click an error or a warning message in the output window to navigate to the affected object in the LL984 Editor.

Errors messages in contrast to warning messages must be fixed, because they inhibit a successful **Build**.

Error Messages

The following is a list of example error messages that **Analyze** will display (E).

Error Message
illegal link
variable '%1' must be located
only coils allowed in last column
only one coil per row
FFB type '%1' isn't supported
last object in row must be coil
maximum column number allowed is '11'
maximum row number allowed is '7'
parameter '%1' expects constant value
illegal constant value '%1': %2
parameter '%1' expects variable
unsupported object
dimension of '%1' is too small, must be at least '%2'
dimension of '%1' must be '%2'
variable '%1' must be writable
parameter of '%1' must be a multiple of '%2'
parameter of '%1' must be a multiple of '%2'
usage of LL984 language is disabled
usage of SKP instruction is disabled
'%1' only allowed in subroutine network
'%1' must be in row1/column1
label '%1' isn't unique
label '%1' doesn't exist
subroutine segment doesn't exist
file '%1' is missing
IMC template '%1' doesn't exist
Non-latched usage of '%1' detected, which is used also as latched

Warning Messages

The following is a list of example warning messages that **Analyze** will display (W).

Warning Message
expression expected (if no actual parameter is assigned to a contact)
variable expected (if no actual parameter is assigned to a coil)

Tolerated Problems

Analyze will tolerate the following problems.

Problem
missing connections (to power rails or other logic elements)
skipping from a subroutine network to a network of another subroutine in the LL984 subroutine segment, if the project setting "SKP enabled" is checked

Network Display

Introduction

The LL984 Editor can display various information concerning variables attached to coils, contacts and function blocks.

This information is displayed above the respective coil, contact or function block node and can be displayed by tooltip, too.

4 Different Views

You can define 4 different views (**View 1** to **View 4**) via **Tools → Project Settings → Program → LL984 → Network Display**.

According to the settings for **View 1** to **View 4** the LL984 Editor will show the following information:

- address (e.g. %M100)
- 984-address (e.g. 400100)
- variable name
- variable comment
- descriptors 1 to 9
- topological address

For detailed information on how to define the different views, please refer to *Project Settings*, section *Program* *(see page 46)*.

View Switching

You can switch between the different views defined, using **CTRL+W** or using the **Change View Mode** button ().

The button's tooltip shows the current view (**View 1, 2, 3 or 4**.

Navigating with the Keyboard

Using the Keyboard

The following keys and key combinations are provided for navigation:

Key combinations	Motion
Left arrow	Moves the gray field in a cell to the left by one cell and selects the contents of that cell. If the first line is reached, the previous line is automatically selected.
Right arrow	Moves the gray field in a cell to the right by one cell and selects the contents of that cell. If the last line is reached, the next line is automatically selected.
Up arrow	Moves the gray field in a cell to the up by one cell and selects the contents of that cell.
Down arrow	Moves the gray field in a cell to the down by one cell and selects the contents of that cell.
Shift+Left arrow	Moves the selected object and the gray field to the left by one cell. This also applies to several selected objects, the gray field must be behind one of the selected objects.
Shift+Right arrow	Moves the selected object and the gray field to the right by one cell. This also applies to several selected objects, the gray field must be behind one of the selected objects.
Shift+Up arrow	Moves the selected object and the gray field up by one cell. This also applies to several selected objects, the gray field must be behind one of the selected objects.
Shift+Down arrow	Moves the selected object and the gray field down one cell. This also applies to several selected objects, the gray field must be behind one of the selected objects.
Home	Positions the gray field in the first column and shows this new location.
End	Positions the gray field in the last column and shows this new location.
Ctrl+Home	Positions the gray field in the upper left hand cell in this network and shows this new location.
Ctrl+End	Positions the gray field in the upper right hand cell in this network and shows this new location.

Key combinations	Motion
Page Up	Scrolls the position of the gray field one page up and shows this new position, as long as the first row is not visible. Displays the previous network/segment, if the first row is already visible. You can also display the previous segment/network via **View → GoTo Previous Section**.
Page Down	Scrolls the position of the gray field one page down and shows this new position, as long as the last row is not visible. Displays the next network/section, if the last row is already visible. You can also display the next section/network via **View → GoTo Next Section**. Using **Page Down** in the last network of a segment, if the last row is already visible, will display a message box asking you to create a new network. Click **Yes** to open the **New Network** dialog. Click **No** to open the next segment/network.
Space bar	Selects or deselects the object in the cells highlighted in gray.
Shift + **Space bar**	The position of the gray field is moved one cell to the right.
Enter	In Insert mode: Inserts the selected object into the currently selected cells highlighted in gray and moves the position of the gray field one cell to the right.
Alt+Enter	Opens the properties dialog box for the selected object/pin.
Tab	Select the next pin in an FFB if the FFB or an FFB pin is selected.
Esc	Activates the Select mode.

NOTE: You can also display a segment/network by double-clicking the respective segment/network in the **Structural View** of the **Project Browser**.

Selecting Objects

Select Mode

Objects are selected in select mode.

Select mode can be activated using:

- the **Edit → Select Mode** menu command
- the  symbol,
  or
- the **Esc** key

The  cursor symbol indicates that select mode is active

Selecting an Object

Using the Mouse	Using the Keyboard
Left-click the object you want to select.	Move the gray field to the cell with the object to be selected, see also Navigating with the Keyboard *(see page 72)*.

Selecting Several Objects

Using the Mouse	Using the Keyboard
1. Click the left mouse button and keep it pressed. 2. Drag the mouse across the objects you want to select. or 1. Left-click the first object you want to select. 2. Press the **Ctrl** key and keep it pressed. 3. Left-click the next object you want to select. 4. Repeat these steps until all desired objects are selected.	Selecting several objects: 1. Move the gray field to the cell with the first object to be selected, see also Navigating with the Keyboard *(see page 72)*. 2. Press the **Shift+Space bar** key combination. **Result:** The object is added to the current selection and the gray field is shifted one cell to the right. 3. Repeat these steps until all desired objects are selected.

Selecting by Rows

Using the Mouse	Using the Keyboard
Selecting the contents of a row: 1. Left-click the number of the row you want to select in the vertical ruler. Selecting the contents of several lines: 1. Left-click the number of the first line you want to select in the vertical ruler. 2. Press the **Shift** key and keep it pressed. 3. Left-click the number of the last line you want to select in the vertical ruler.	-

Selecting by Columns

Using the Mouse	Using the Keyboard
Selecting the contents of a column: 1. Left-click the number of the column you want to select in the horizontal ruler. Selecting the contents of several columns: 1. Left-click the number of the first column you want to select in the horizontal ruler. 2. Press the **Shift** key and keep it pressed. 3. Left-click the number of the last column you want to select in the horizontal ruler.	-

Selecting All

Using the Mouse	Using the Keyboard
Use the **Edit → Select All** menu command.	Press the **Ctrl+A** key combination.

Deselecting Objects

Using the Mouse	Using the Keyboard
Left-click an empty space in the section.	Move the gray field.

Deleting, Cutting, Copying, Pasting and Moving Objects

Deleting Objects

Using the Mouse	Using the Keyboard
1. Select *(see page 74)* the object to be deleted. 2. Use the **Edit → Delete** menu command.	Deleting the selected object: 1. Select *(see page 74)* the object to be deleted. 2. Press the **Del** key. Deleting the object left of the gray field: 1. Press the **Backspace** key.
Result: The selected object is deleted. The actual parameters are deleted together with the object, even if they were not selected explicitly. Boolean links are only deleted if they are selected explicitly.	

Cutting Objects

Using the Mouse	Using the Keyboard
1. Select the object to be cut. 2. Use the **Edit → Cut** menu command. or Use **Cut** from the context menu (right-click).	1. Select the object to be cut. 2. Press the **Ctrl+X** key combination.
Result: The selected object is cut from the section and copied to the clipboard. This is also the case for the actual parameters.	

The cut object can be inserted in any other position (also in another LL984 network).

Copying Objects to the Clipboard

Using the Mouse	Using the Keyboard
1. Select the object to be copied. 2. Use the **Edit → Copy** menu command. or Use **Copy** from the context menu (right-click).	1. Select the object to be copied. 2. Press the **Ctrl+C** key combination.
Result: The selected object is copied to the clipboard. This is also the case for the actual parameters.	

The cut object can be inserted in any other position (also in another LL984 network).

Pasting Objects from the Clipboard

Using the Mouse	Using the Keyboard
1. Use the menu command **Edit** → **Insert**. or Use **Paste** from the context menu (right-click). 2. Left-click the target position.	1. Move the cursor to the target position pressing **Ctrl** and using the arrow keys. 2. Press the **Ctrl+V** key combination. 3. Press **Enter**.

NOTE: (for pasting contacts and coils) If the target position already contains a contact or coil, the existing object is overwritten by the new object.

NOTE: The following is the behavior of the **Copy**, **Cut** and **Paste** on a function block instance. It only applies to graphical languages as **FBD**, **LD** and **LL984**.

- Using the **Paste** function after a **Copy** of an object:
  As a result of a **Copy**, a new function block instance (**FBI name**) is used. The **Paste** function creates a new FBI whenever it is repeated. Accordingly, the FBI name is incremented.
- Using the **Paste** function after a **Cut** of an object:
  As a result of a **Cut**, the same instance of the function is used. The **Paste** function uses the same FBI whenever it is repeated. Accordingly, the FBI is identical.

NOTE: Using same instance of a function block is not recommended because it can lead to incorrect execution of the function blocks. A function block instance uses contextual variables that are owned by each instance usage and cannot be applied to any other. For instance, each current value is specific to one timer instance and one timer usage (Current value of a timer named TON_1 cannot be shared with another timer named TON_2).

Moving Objects

Using the Mouse	Using the Keyboard
1. Select the object to be moved. 2. Position the mouse pointer on the selected object. (On one of the selected objects if several are selected). **Result:** The mouse pointer changes its symbol to  . 3. Click the left mouse button and keep it pressed. **Result:** The mouse pointer changes its symbol to  . 4. Drag the object to the new position and release the mouse button.	1. Position the gray field on the object to be moved. 2. Press the **Space bar**. 3. Press the **Shift** key and keep it pressed. 4. Move the object to the target position using the arrow keys.
Result: The selected object is moved from its original position to the target position. The object's actual parameters are moved together with the object. Boolean links are only kept for horizontal movement operations. **NOTE:** A moving of FFBs to cells that are already occupied (e.g. by contact, coil, other FFB, boolean link), is not possible. **NOTE:** The move operation is also possible across sections between different opened LL984 networks. **NOTE:** When objects are moved beyond the visible area of the editor window, automatic scrolling of the window will not begin until you reach the edge of the window with the mouse pointer, not as soon as the objects are moved beyond the edge.	

Copying Objects via Drag-and-Drop

Using the Mouse	Using the Keyboard
1. Select the object to be copied. 2. Position the mouse pointer on the selected object. **Result:** The mouse pointer changes its symbol to  . 3. Click the left mouse button and keep it pressed. 4. Press the **Ctrl** key and keep it pressed. **Result:** The mouse pointer changes its symbol to  . 5. Drag the object to the new position. **NOTE:** This is also possible across sections between different opened LL984 networks. 6. Release the mouse button. **Result:** A copy of the selected object is pasted at the target position. The object's actual parameters (variable/address) are copied together with the object. A new instance is automatically created for FFBs.	-

Displaying the Properties

Displaying the Object Properties

You have the following options to display the object properties dialog:

- Double-click the object.
- Select *(see page 74)* the object and use the **Edit → Properties...** dialog.
- Select the object and use **Properties** from the context menu (right-click).
- Select the object and then press the **Alt+Enter** key combination.

When the properties dialog is called without an object being selected, the Network Properties *(see page 40)* dialog is opened.

If several objects are selected, the commands do not work.

Displaying the Data Properties

You have the following options to display the **Data Properties** dialog *(see page 127)*:

- Via the LL984 network:
  Select an object and use **Data Properties** from the context menu or press the **Ctrl+Enter** key combination.
- Via the Data Editor
  Select the line of the respective object in the Data Editor and use **Data Properties** from the context menu or press the **Ctrl+Enter** key combination or use the **Edit → Data Properties** dialog.

Inserting and Deleting Rows/Columns

Introduction

Each network contains a matrix of 7 rows and 11 columns.

But via **Insert/Delete row** and **Insert/Delete column** you can shift rows downwards/upwards and shift columns to the right/left.

Rules for Inserting Rows/Columns

The following rules apply when inserting rows/columns.

- The new row is inserted above the first selected row.
  The content of the following rows will be shifted to the bottom.
- The new column is inserted left of the first selected column.
  The content of the following columns will be shifted to the right.
- If there are no free rows/columns at the bottom/right of the network, the menu items will be disabled.
- If the new row would intersect a middle or a bottom node of an FFB, the menu items will be disabled.

Rules for Deleting Rows/Columns

The following rules apply when deleting rows/columns.

- You can not delete a row/column containing any object.
- If there are no free rows/columns at the current position, the menu items will be disabled.

Inserting Rows

Step	Action
1	Select the desired row in the vertical ruler (click the row number).
2	Use the **Insert row** menu command from the context menu of the vertical ruler or use the **Edit → Insert row** menu command. **Result:** A row is inserted above the selected row.

Inserting Columns

Step	Action
1	Select the desired column in the horizontal ruler (click the column number).
2	Use the **Insert column** menu command from the context menu of the horizontal ruler or use the **Edit → Insert column** menu command. **Result:** A column is inserted left of the selected column.

Deleting Rows

Step	Action
1	Select the desired row in the vertical ruler (click the row number).
2	Use the **Delete row** menu command from the context menu of the vertical ruler or use the **Edit → Delete row** menu command. **Result:** The row is deleted. The rows below are shifted on row up. At the bottom an empty row is inserted.

Deleting Columns

Step	Action
1	Select the desired column in the horizontal ruler (click the column number).
2	Use the **Delete column** menu command from the context menu of the horizontal ruler or use the **Edit → Delete column** menu command. **Result:** The column is deleted. The columns from the right are shifted one column to the left. At the right side of the grid an empty column is inserted.

Section 6.2
Editing Contacts

Overview

This section describes how to edit contacts in the LL984 programming language.

What Is in This Section?

This section contains the following topics:

Topic	Page
Selecting Contacts	84
Placing Contacts	86
Properties Dialog for Contacts	90

Selecting Contacts

Introduction

A contact is an LL984 element that transfers a status on the horizontal link to its right side. This status comes from the Boolean AND link of the status of the horizontal link on the left side with the status of the relevant Boolean current parameter.

Information Concerning Variables

The LL984 Editor can display various information concerning variables attached to coils, contacts and function block nodes.

This information can be displayed by tooltip, too.

For further information on how the variables attached to contacts are displayed, please refer to *Network Display* *(see page 71)*.

Contact Types

Contact Type	Description	Call via Menu Command	Call via Icon	Call via Key
Normally open contact	For normally open contacts, the status of the left link is copied to the right link, if the status of the relevant Boolean actual parameter is 1. Otherwise, the status of the right link is 0.	**Edit → New → Normally open contact** or **Normally open contact** from the context menu		**F3**
Normally closed contact	For normally closed contacts, the status of the left link is copied to the right link, if the status of the relevant Boolean actual parameter is 0. Otherwise, the status of the right link is 0.	**Edit → New → Normally closed contact** or **Normally closed contact** from the context menu		**Shift+F3**
Positive transition-sensing contact	With contacts for detection of positive transitions, the right link for a program cycle is 1 if a transfer of the relevant actual parameter goes from 0 to 1 and the status of the left link is 1 at the same time. Otherwise, the status of the right link is 0. Also see Edge Recognition *(see Unity Pro, Program Languages and Structure, Reference Manual)*.	**Edit → New → Positive transition-sensing contact** or **Positive transition-sensing contact** from the context menu		**Ctrl+F3**

Contact Type	Description	Call via Menu Command	Call via Icon	Call via Key
Negative transition-sensing contact	With contacts for detection of negative transitions, the right link for a program cycle is 1 if a transfer of the relevant actual parameter goes from 1 to 0 and the status of the left link is 1 at the same time. Otherwise, the status of the right link is 0. Also see Edge Recognition *(see Unity Pro, Program Languages and Structure, Reference Manual)*.	**Edit → New → Negative transition-sensing contact** or **Negative transition-sensing contact** from the context menu		**Ctrl** + **Shift** + **F3**

Placing Contacts

Introduction

Contacts can be placed in any free cell except cells directly on the right power rail.

If a contact is placed in a cell that was previously occupied by a contact, coil, a Boolean link or a label, the cell contents are replaced by the new contact.

If a contact is placed in a cell that is already occupied by another object (e.g. FFBs), an error message is returned.

The following are permitted as actual parameters for contacts:

- Boolean variables
- Boolean literals (`0`, `1`, `FALSE`, `TRUE`)
- Boolean addresses (topological addresses or symbolic addresses)
- ST expressions that return a Boolean result (e.g. `VarA > VarB`)

  ST expressions as formal parameters on contacts are an extension of IEC 61131-3 and must be explicitly enabled via the **Tools → Project Settings → Program → Languages → Common** by activating the **Usage of ST expressions** check box.

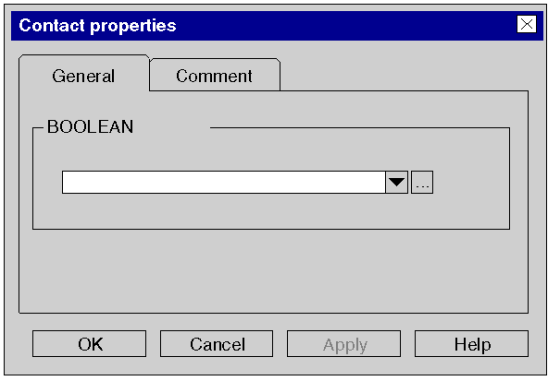
Automatically Assign a Variable

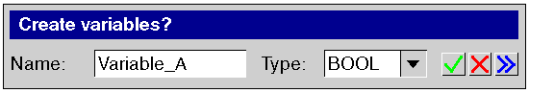
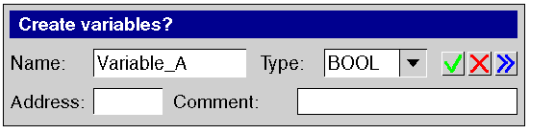
If via **Tools → Options → Data and Languages → Languages** the **Automatically assign a variable to a new graphical object** check box is activated, then the corresponding properties dialog is automatically opened when an object is placed. The procedures explained here are used when the check box is **deactivated**.

Placing Contacts

Step	Action
1	Select the desired contact, see also *Selecting Contacts* *(see page 84)*.
2	Click the target cell in the LL984 section. or Use the **Arrow** keys to move the gray field to the target position, and press **Enter**. **Result:** The selected contact is inserted.
3	To place further contacts of the same type: ● Click the target cell in the LL984 section. or Use the **Arrow** keys to move the gray field to the target position and press **Enter**. ● To return to selection mode: Press the **Esc** key. ● To insert other objects: Select the object you wish to insert.

Assigning Actual Parameters

Step	Action
1	Open the properties dialog *(see page 80)* for the contact. **Result**: The contacts properties dialog is opened. Example: 
2	You have the following options to enter the actual parameters: • You can enter %Mx (%Ix) and confirm using **Enter**. • You can enter the actual parameter and confirm using **Enter**. • You can select the actual parameter from the list of recently used names using the ▼ symbol and confirm using **Enter**. or • Using the button **...** you can open a **Data Selection** dialog and confirm the selected variables there with **OK**.

Step	Action
3	Confirm the selected variable with **OK**. **Result:** • If the selected variable is declared, it is enabled in the section. A syntax and semantics check *(see page 68)* is performed and the procedure ends here. Variable_A • If the variable is not declared, a **Create variable?** dialog opens. In this case, continue with the following steps of this procedure.  **NOTE:** If you wish to declare the variable later, close the dialog using the symbol. In this case, the variable name is enabled in the section, but not declared. **NOTE:** Entering a binary address (%M1 / %I1) opens the **Create variable** dialog proposing a variable name (_000001 / _100001) that can be modified. If there is already a variable associated to %M1 / %I1 it will be reused as proposal.
4	If you do not want to assign the variable an address or comment, confirm using **Enter** or the symbol. **Result:** The variable is declared and enabled in the section. A syntax and semantics check is performed and the procedure ends here. Example: Variable_A
5	If you do want to assign the variable an address and/or comment, use the symbol for obtaining an advanced dialog and enter the address and/or comment. 
6	Confirm with **Enter** or the symbol. **Result:** The variable is declared and enabled in the section. A syntax and semantics check is performed immediately after the statement is entered. Example: Variable_A

Usage of ST Expressions

To enter ST expressions for contacts the same rules apply as for entering ST expressions for FFB pins *(see Unity Pro, Operating Modes)*.

Properties Dialog for Contacts

Calling the Properties Dialog

See Displaying the Properties *(see page 80)*.

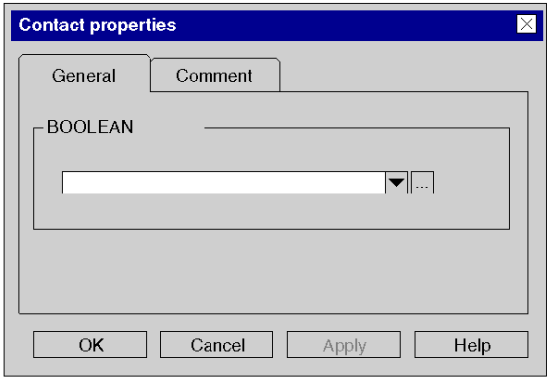
Structure of the Properties Dialog

The contact properties dialog consists of 2 tabs:

- **General**
  In this tab, you can enter the actual parameter *(see page 87)* of the contact.
- **Comment**
  In this tab, you can enter a comment about the contact.

General Tab

Representation of the **General** tab



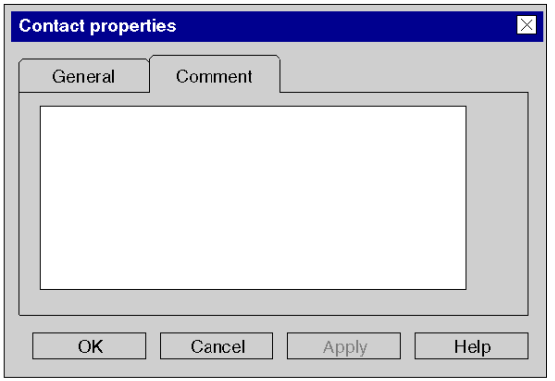
Elements of the **General** tab

Element	Description
BOOLEAN Expression	Enter the name of the actual parameter in this text box. You have the following options: ● You can directly enter the name of the variable/address or paste it from the clipboard. ● You can select the name of the variable from the list of recently used names using the ▼ symbol. or ● Use the command button **...**to open a **Data Selection** dialog *(see Unity Pro, Operating Modes)*. The actual parameters which are permitted are: ● Boolean variables ● Boolean constants ● Boolean addresses (topological addresses or symbolic addresses) ● ST expression delivering a Boolean result (e.g. `BoolVar1 OR BoolVar1`) ● Literal (`1` or `0` or. `TRUE` or `FALSE`)
OK	Use this command button to accept the entries and close the dialog.
Apply	Use this command button to accept the entries without closing the properties dialog.

NOTE: Entering a binary address (%M1 / %I1) opens the **Create variable?** dialog proposing a variable name (_000001 / _100001) that can be modified. If there is already a variable associated to %M1 / %I1 it will be reused as proposed.

Comment Tab

Representation of the **Comment** tab



Elements of the **Comment** tab

Element	Description
Text box	Enter a comment about the contact. This comment is displayed as a tooltip when the cursor is placed over the contact. It is independent from the variable comment that can be assigned to the variable in the Data Editor.
OK	Use this command button to accept the entries and close the dialog.
Apply	Use this command button to accept the entries without closing the properties dialog.

Section 6.3
Editing Coils

Overview

This section describes how to edit coils in the LL984 programming language.

What Is in This Section?

This section contains the following topics:

Topic	Page
Selecting Coils	94
Placing Coils	96
Property Dialog for Coils	99

Selecting Coils

Introduction

A coil is an LL984 element which transfers the status of the horizontal link on the left side, unchanged, to the horizontal link on the right side. The status is saved in the respective Boolean actual parameter. Coils normally follow contacts or FFBs, (functions and function blocks), but they can also be followed by contacts.

Information Concerning Variables

The LL984 Editor can display various information concerning variables attached to coils, contacts and function block nodes.

This information can be displayed by tooltip, too.

For further information on how the variables attached to coils are displayed, please refer to *Network Display* *(see page 71)*.

Coil Types

⚠ CAUTION

UNINTENDED EQUIPMENT OPERATION

Be sure of the output states after a PLC start when using latched and non-latched coils.

Before using non-latched or latched coils, make sure that you completely understand the different behavior of non-latched/latched coils.

- **Non-latched Coils**
  A reset of the coil will be performed during warm-start and after STOP/RUN of the PLC. Reset during warm-start works for %M addresses and %M-located-variables attached only.
- **Latched Coils**
  In the first scan latched coils have the status of last scan before warm-start.

Failure to follow these instructions can result in injury or equipment damage.

Coil Type	Description	Call via Menu Command	Call via Icon	Call via Keyboard
Coil (non-latched)	With coils, the status of the left link is copied to the relevant Boolean actual parameter and the right link. For further information see below.	**Edit → New → Coil** or **Coil** from the context menu		**F5**
Latched coil	With latched coils, the status of the left link is copied to the relevant Boolean actual parameter and the right link. For further information see below.	**Edit → New → Latched coil** or **Latched coil** from the context menu		**Shift+F5**

PLC Start Behavior of Coils

- **Non-latched Coils**
  A reset of the coil will be performed during warm-start and after STOP/RUN of the PLC. Reset during warm-start works for %M addresses and %M-located-variables attached only.
- **Latched Coils**
  Latched coils will maintain their state after a Stop/Start of the PLC and on warm-start. In the first scan latched coils have the status of last scan before warm-start.
  This is the same behavior as normal coils show in IEC Ladder Diagram language.

Detailed information:

- For details when a warm-start is performed please see *Processing of Power Outage and Restoral for Premium/Quantum PLCs (see Unity Pro, Program Languages and Structure, Reference Manual)*.
  Especially for the case of a Power Cycle this depends on the position of the PLC's key switch (**MemPrt**/**Start**/**Stop**).
  - For firmware version 2.8:
    Latched coils are **not** maintained if the key switch is in the Start position.
  - For firmware version 2.6:
    Latched coils **are** maintained regardless of the key switch position.
- For details regarding the PLC's key switch refer to *Key Switches (see Quantum with Unity Pro, Hardware, Reference Manual)*.
  For details regarding Cold Start and warm-start processing refer to *Processing on Cold Start for Premium/Quantum PLCs (see Unity Pro, Program Languages and Structure, Reference Manual)* and *Processing on Warm Restart for Premium/Quantum PLCs (see Unity Pro, Program Languages and Structure, Reference Manual)*.

Non-latched Coil Restriction

The following restriction applies for non-latched coils:

- If an address (or a variable located on this address) is attached to a normal LL984 coil (non-latched) and this address has other (latched) write usages somewhere else in the program (for example an assignment in ST/FBD/LD or a latched coil in an LL984 network), this will lead to the following analyze message:
  `Non-latched usage of '%1' detected, which is used also as latched.`
  This message will be reported for the non-latched coil(s) usage in LL984 networks only.
  To help avoid this restriction, start a search for getting the occurrences of this conflicting address.

Latched Coil Representation

A latched coil is represented by a circle with an L or an M in the middle.

Whether an **L** or an **M** will be displayed inside a latched coil can be define via **Tools** → **Options** → **Data and Languages** → **LL984** → **Show latched coil as**. Please refer to *Data and Languages (see page 51)*.

Placing Coils

Introduction

For placing coils the following restrictions apply:

- Only 1 coil can be placed per row.
- No other object can be placed to the right of a coil. The coil is always the last object in a row.
- If a coil is placed in a cell that is already occupied by another object (e.g. FFBs), an error message is returned.

A placed coil automatically creates a connection with its neighboring objects on the left if they are of the `BOOL` data type, even if free cells are between them.

The following are permitted as actual parameters for coils:

- Boolean variables
- Boolean addresses (topological addresses or symbolic addresses)

Show Coils in Last Column

Using the main menu **View** you can activate the **Show coil in last column** setting.

If this setting is activated, each coil will be displayed in the last column with a dotted line between its "real" location cell and the last column.

Otherwise each coil will be displayed at its "real" location cell.

NOTE: Activating this setting will switch the LL984 Editor to **Read only** mode. No modifications are allowed inside the LL984 Editor. To leave the **Read only** mode, just deactivate this setting.

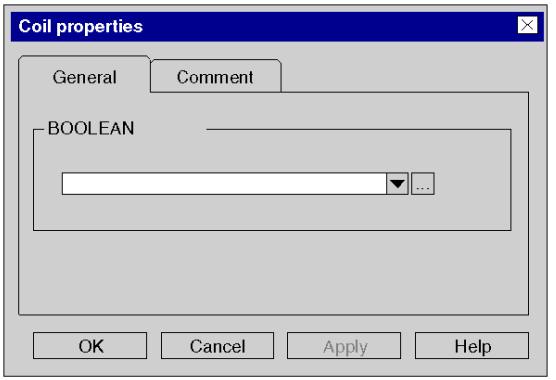
Automatically Assign a Variable

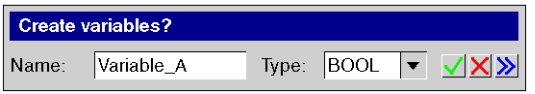
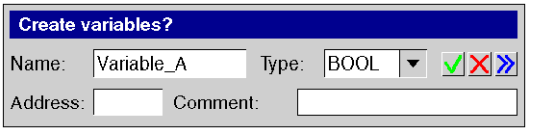
If via **Tools → Options → Data and Languages → Languages** the **Automatically assign a variable to a new graphical object** check box is activated, then the corresponding properties dialog is automatically opened when an object is placed. The procedures explained here are used when the check box is **deactivated**.

Placing Coils

Step	Action
1	Select the desired coil. See also Selecting Coils *(see page 94)*.
2	Click the target cell in the LL984 section. or Use the **Arrow** keys to move the gray field to the target position, and press **Enter**. **Result:** The selected coil is inserted.
3	To place further coils of the same type: ● Click the target cell in the LL984 section. or Use the **Arrow** keys to move the gray field to the target position and press **Enter**. ● To return to selection mode: Press the **Esc** key. ● To insert other objects: Select the object you wish to insert.

Assigning Actual Parameters

Step	Action
1	Open the properties dialog *(see page 80)* for the coil. **Result**: The coils properties dialog is opened. Example: 

Step	Action
2	You have the following options to enter the actual parameters: ● You can enter %Mx (%Ix) and confirm using **Enter**. ● You can enter the variable/address name and confirm using **Enter**. ● You can select the variable name from the list of recently used names using the ▾ symbol and confirm using **Enter**. or ● Using the button **...** you can open a **Data Selection** dialog and confirm the selected variables there with **OK**.
3	Confirm the selected variable with **OK**. **Result:** ● If the selected variable is declared, it is enabled in the section. A syntax and semantics check *(see page 68)* is performed and the procedure ends here. ● If the variable is not declared, a **Create variable?** dialog opens. In this case, continue with the following steps of this procedure.  **NOTE:** If you wish to declare the variable later, close the dialog using the ✕ symbol. In this case, the variable name is enabled in the section, but not declared. **NOTE:** Entering a binary address (%M1 / %I1) opens the **Create variable?** dialog proposing a variable name (_000001 / _100001) that can be modified. If there is already a variable associated to %M1 / %I1 it will be reused as proposal.
4	If you do not want to assign the variable an address or comment, confirm using **Enter** or the ✓ symbol. **Result:** The variable is declared and enabled in the section. A syntax and semantics check is performed and the procedure ends here.
5	If you do want to assign the variable an address and/or comment, use the » symbol for obtaining an advanced dialog and enter the address and/or comment. 
6	Confirm with **Enter** or the ✓ symbol. **Result:** The variable is declared and enabled in the section. A syntax and semantics check is performed.

Property Dialog for Coils

Calling the Properties Dialog

See Displaying the Properties *(see page 80)*.

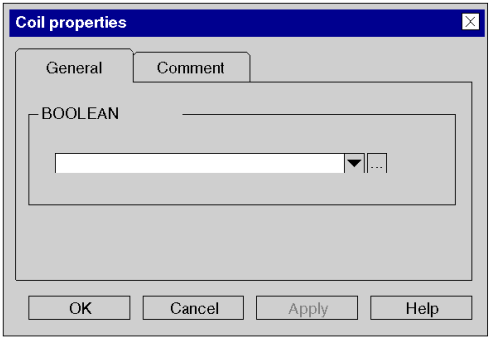
Structure of the Properties Dialog

The coil properties dialog consists of 2 tabs:

- **General**
  In this tab, you can enter the actual parameter *(see page 97)* of the coil.
- **Comment**
  In this tab, you can enter a comment about the coil.

General Tab

Representation of the **General** tab

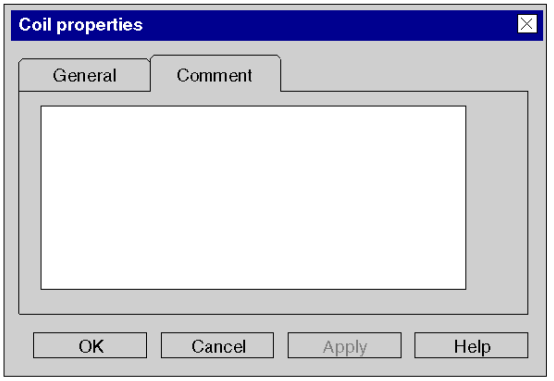


Elements of the **General** tab

Element	Description
BOOLEAN Expression	Enter the name of the actual parameter in this text box. You have the following options: ● You can directly enter the name of the variable/address or paste it from the clipboard. ● You can select the name of the variable from the list of recently used names using the ▼ symbol. or ● Use the command button **...** to open a **Data Selection** dialog *(see Unity Pro, Operating Modes)*. The actual parameters which are permitted are: ● Boolean variables ● Boolean addresses (topological addresses or symbolic addresses)
OK	Use this command button to accept the entries and close the dialog box.
Apply	Use this command button to accept the entries without closing the properties dialog box.

Comment Tab

Representation of the **Comment** tab



Elements of the **Comment** tab

Element	Description
Text box	Enter a comment about the coil. This comment is displayed as a tooltip when the cursor is placed over the coil. It is independent from the variable comment that can be assigned to the variable in the Data Editor.
OK	Use this command button to accept the entries and close the dialog box.
Apply	Use this command button to accept the entries without closing the properties dialog box.

Section 6.4
Editing Function Blocks

What Is in This Section?

This section contains the following topics:

Topic	Page
Editing Function Blocks	102
Using Mnemonics	105

Editing Function Blocks

Introduction

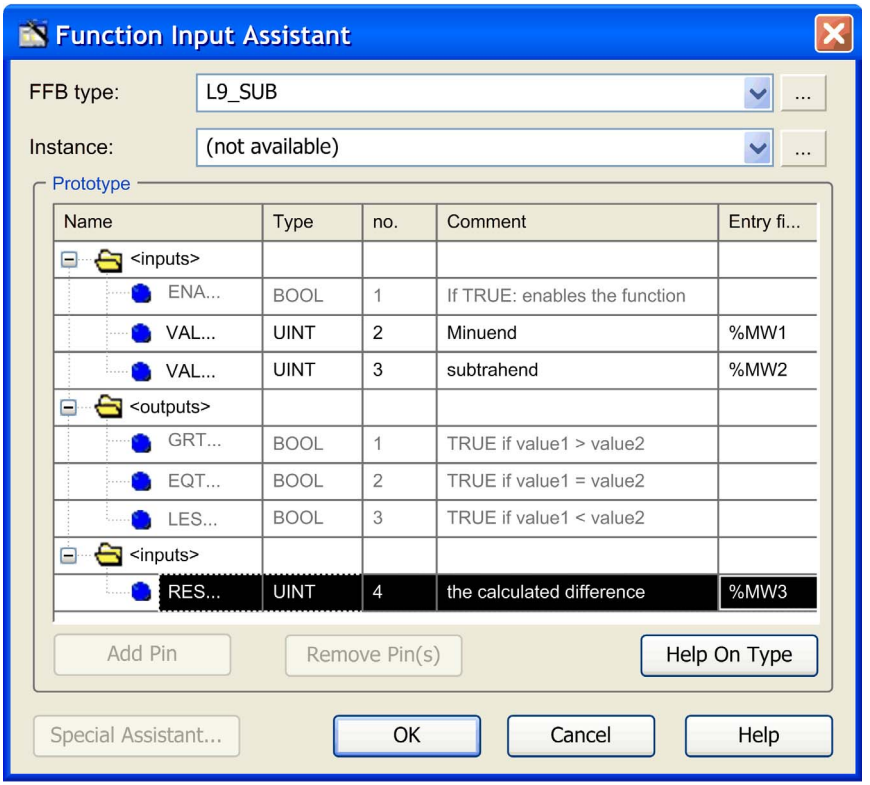
You can insert function blocks using one of the following features:

- **LL984 Instruction Bar**
- **Mnemonics**
- **Data Selection**
- **FFB Input Assistant**.
- **Types Library Browser**,
- **Copy/Paste**

Inserting a Function Block

Step	Action
1	In the LL984 Editor right-click and select from the context menu either ● **Data Selection** (**Ctrl+D**) or ● **FFB Input Assistant** (**Ctrl+I**). You can also insert a function block using: ● drag-and-drop from the **Types Library Browser** (**Alt+3**), . ● copy/paste ● LL984 Instruction Bar ● Mnemonics
2	Enter the complete name of the function block (e.g. L9_SUB).
3	Confirm with **Enter**.
4	Click the cell where you want to insert the function block. **Result**: The function block is inserted (if allowed at this position).

Representation of the FFB Input Assistant



You can also see the **Comments** displayed in the dialog above as a tooltip when the cursor is placed over the respective pin of the function block.

Editing Values

To open the **FFB Input Assistant** for an already inserted function block, right-click the function block and select **FFB Input Assistant** from the context menu.

Click the **Entry Field** and enter a value.

You are only allowed to edit the **Entry fields** displayed in black. Only the nodes (top, middle, bottom) of a function block can be edited.

You can not edit the **Entry fields** displayed in grey (disabled).

Restrictions

For inserting function blocks the following restrictions apply:

- Only LL984 library function blocks can be inserted in LL984 networks. If you try to insert a function block of another library a message will inform you, that this is not possible.
- You can not insert self-programmed DFBs in LL984 networks.
- LL984 library function blocks can not be inserted in IEC languages sections (FBD, LD, ST, IL). SFC does not use function blocks.
  - FBD and LD editors will refuse insertion and will display a message that insertion is not possible.
  - For textual languages (ST, IL) the call of an LL984 library function block will be detected during **Analyze**.

Using Mnemonics

Introduction

Instead of inserting LL984 objects via toolbar, **FFB Input Assistant**, **Data Selection** or drag-and-drop, you can use so called **Mnemonics**.

Mnemonics Option

Open the **Tools → Options** dialog.

Under **Data and Languages → Languages → LL984 → Mnemonics** the **Property label** column lists the contacts and coils and an assortment of frequently used functions blocks available for LL984 networks.

In the **Property value** column you can enter a mnemonic for each listed element.

A mnemonic can consist of up to 4 alphanumeric signs.

Using Mnemonics

As you start typing text, while the current cell marker is on an empty cell in the LL9894 Editor, a small edit box opens in the editor.

Entering a mnemonic (first key typed will be taken over into the edit box) and pressing the **Enter** key will insert the corresponding LL984 element at the current cell.

If the cells needed to insert the element are already occupied, a message box will inform you and the element will not be inserted.

Section 6.5
Editing Links

Overview

This section describes how to edit links in the LL984 programming language.

What Is in This Section?

This section contains the following topics:

Topic	Page
Selecting Links	107
Combining Links	108
Placing Links	109
Editing Links	111

Selecting Links

Introduction

Links are connections between LL984 objects (contacts, coils, FFBs etc.).

Boolean connections comprise one or more cell-segments that connect Boolean objects (contacts, coils) with one another.

- **Boolean Connection** (horizontal connection)
  This horizontal connection enables series switching of contacts and coils.
  The cell-segments of this connection can be created individually or as a complex connection comprising several cell-segments.
- **Boolean Link** (horizontal link)
  This horizontal link automatically creates a connection between neighboring Boolean objects to the left and right or between an object and the power rail.
- **Vertical Connection**
  This vertical connection enables parallel switching of contacts and coils.

Link Types

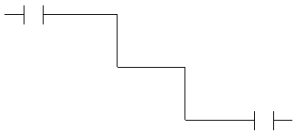
Link Type	Description	Call via Menu Command	Call via Icon	Call via Key
Horizontal connection	horizontal connection, 1 cell wide	**Edit → New → Boolean Connection** or **Boolean Connection** from the context menu		**F7**
Horizontal link	A horizontal link creates a connection between neighboring Boolean objects to the left and right. If there are no neighboring Boolean objects available, a connection to the power rail is made.	**Edit → New → Boolean Link** or **Boolean Link** from the context menu		**Alt+F6**
Vertical connection	vertical connection, 1 cell high	**Edit → New → Vertical Connection** or **Vertical Connection** from the context menu		**Shift+F7**

Combining Links

Boolean Links

Horizontal and vertical Boolean links can be combined in any way.

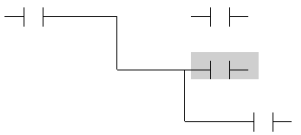
Example:



Contacts and Coils

Contacts and coils link automatically to existing horizontal or vertical links if no free cells are found between them.

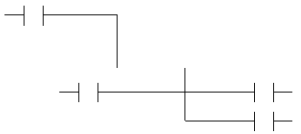
Example:



Crossing Boolean Links

If 2 Boolean links cross each other they are automatically connected. Since Boolean links may not cross each other there is no special label for them.

Example:



Placing Links

Placing Horizontal Connections

Process the following steps to place a horizontal link.

Step	Action
1	Select the horizontal connection (**Boolean Connection**). See also Selecting Links *(see page 107)*.
2	Click the target cell in the LL984 section. or Use the **Arrow** keys to move the gray field to the target position and press **Enter**. **Result:** The link is inserted.
3	To place further links of the same type: ● Click the target cell in the LL984 section. or Use the **Arrow** keys to move the gray field to the target position and press **Enter**. ● To return to selection mode: Press the **Esc** key. ● To insert other objects: Select the object you wish to insert.

Placing Horizontal Links

Process the following steps to place a horizontal link.

Step	Action
1	Select the horizontal link (**Boolean Link**). See also Selecting Links *(see page 107)*.
2	Click the target cell in the LL984 section. or Use the **Arrow** keys to move the gray field to the target position and press **Enter**. **Result:** The link is inserted and a connection is created to the neighboring Boolean objects to the left and right. If there are no neighboring Boolean objects available, a connection to the power rail is made.
3	To place further links of the same type: ● Click the target cell in the LL984 section. or Use the **Arrow** keys to move the gray field to the target position and press **Enter**. ● To return to selection mode: Press the **Esc** key. ● To insert other objects: Select the object you wish to insert.

Placing Vertical Connections

On the right side of the cells, there is space to create a vertical connection to the cell below or above. Process the following steps to place a vertical connection.

Step	Action
1	Select the vertical connection (**Vertical Connection**). See also Selecting Links *(see page 107)*.
2	Click the object which should be connected with the cell below. or Use the **Arrow** keys to move the gray field to the target position and press **Enter**. **Result:** The link is inserted.
3	To place further links of the same type: ● Click the target cell in the LL984 section. or Use the **Arrow** keys to move the gray field to the target position and press **Enter**. ● To return to selection mode: Press the **Esc** key. ● To insert other objects: Select the object you wish to insert.

Editing Links

Introduction

If you combine several horizontal links this will result in 1 horizontal link.

If you combine several vertical links this will result in 1 vertical link.

If you combine a horizontal and a vertical link this will result in a link made of 2 objects, a horizontal and a vertical link, that can be selected independent from each other.

Deleting, Cutting, Copying, Pasting and Moving Links

You can delete, cut, copy, paste and move links.

For further information please refer to Deleting, Cutting, Copying, Pasting and Moving Objects *(see page 76)*.

Modifying the Size of Links

Step	Action
1	Select a link. **Result:** Size handles are displayed at the beginning and at the end of the link.
2	Click a size handle with the left mouse button and keep it pressed.
3	Drag the size handle to the desired new end position of the link and release the mouse button.

Section 6.6
Editing Equation Network Blocks

Overview

This section describes how to edit Equation Network Blocks in the LL984 programming language.

What Is in This Section?

This section contains the following topics:

Topic	Page
Editing Equation Network Blocks	113
Editing the ST Assignment Expression	115

Editing Equation Network Blocks

Introduction

Equation networks in the LL984 Editor are provided as **Equation Network Blocks**.

Equation network blocks offer an opportunity to program complex mathematical functions, with values stored in memory words.

An **Equation Network Block** contains an assignment expression written in ST language.

In general you can use the features provided by ST language as long as it is a valid ST assignment expression.

Please refer to the Structured Text (ST) chapter *(see Unity Pro, Program Languages and Structure, Reference Manual)* and the ST Editor chapter *(see Unity Pro, Operating Modes)*.

Using the Equation Network Block

You can insert an **Equation Network Block** using the respective toolbar button, via menu (**Edit** → **New** → **Equation Network block**) or via the editors context menu (right mouse click).

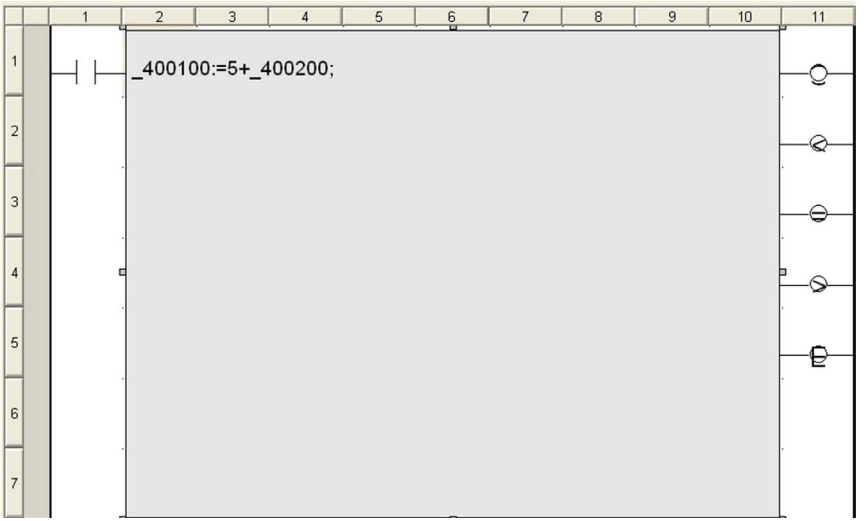
An **Equation Network Block** can only be inserted in an LL984 network at row 1, column 2. The block is 7 rows high and 9 columns wide.

On the left side of an equation network block there is an **Enable** input pin, where you can connect a contact or a boolean connection.

On the right side there are 5 output pins (**OK**, **Lower**, **Equal**, **Greater**,**Error**), where you can connect 5 normally open coils. After inserting these coils are displayed with symbols inside according to the pin names (**O**, **<**, **=**, **>**, **E**).

Representation

Representation of the **Equation Network Block**



Items of the **Equation Network Block**

Item	Description
input pin **Enable**	you can connect a contact or a boolean connection to enable the block
equation network block	expression written in ST language
output pins	**OK (O)**: set when the equation is solved without problems
	Lower (<): set when the equation result is less than zero
	Equal (=)set when the equation result is equal to zero
	Greater (>)set when the equation result is greater than zero
	Error (E): set when a problem occurred during solving the equation (see table below)

Problems During Solving

Problem	Description
invalid operation	An internal message generated by the math coprocessor.
overflow	A value is too large to be represented in its specified data type.
underflow	A number is too small to be represented in FP format (for floating point data only).
divided by 0	The variable, constant, or result of a function directly to the right of a / operator has a value of zero.

Editing the ST Assignment Expression

Introduction

After inserting an **Equation Network Block** double-click the displayed question marks (**?????**) to enter the ST assignment expression.

Confirming with **Enter** will start an analysis and the found detected problems will be displayed in a tooltip.

Animation of the memory words used inside the ST assignment expression can be seen in a **Variable Window**. Please refer to *Variable Display Window (see Unity Pro, Operating Modes)*.

Memory Addresses

A memory address entered in the ST assignment expression must start with one of the following symbols, specifying the address type:

- %M (memory bit)
- %I (discrete input)
- %IW (input word)
- %MW (memory word)

The address type must be followed by an address number (no leading zeros required).

Creating Variables Automatically

After confirming the ST assignment expression with **Enter**, for every entered address a variable will be created automatically, which is used in the equation network.

Examples

Address	Variable
%M1	_000001
%I1	_100001
%IW1	_300001
%MW1	_400001

The variable type is derived from its address suffix.

Suffix		Variable Type
no suffix	%Mx, %Ix	EBOOL
	%IWx, %MWx	UINT
I		INT
DI		DINT
UD		UDINT

Suffix	Variable Type
R	REAL
W	WORD

The variable is mapped to the associated internal located memory.

For example: %IW1DI becomes the variable `_300001_DI:DINT@%IW1`.

Constants

Constants in equation networks have the same syntax as in other IEC editors (that is without a leading # character).

Operators

Operators and their precedence

Operation	Symbol	Precedence
Parentheses	(expression)	HIGHEST
Function evaluation	identifier (argument list)	-
Negation	-	-
Complement	~	-
Exponentiation	**	-
Multiply	*	-
Divide	/	-
Add	+	-
Subtract	-	-
Comparison	< , > , <= , >=	-
Equality	=	-
Inequality	<>	-
Boolean AND	&	-
Boolean Exclusive OR	^	-
Boolean OR	|	-
Assignment	:=	LOWEST

The operators **, *, /, +, -, <<, >>, <, >, <=, >=, =, <>, &, ^ and | are supported as they are implemented now in Unity Pro ST syntax. That is not all data type combinations for operand types and the result type are supported.

NOTE: Unity Pro only supports a floating point value for the ** operator (EXPT-functions).

NOTE: Bitwise logical AND, OR or XOR operations work like in ST syntax.

Ternary Conditional

Ternary conditional (C?t:f), in contrast to legacy LL984, is not available in Unity Pro ST syntax.

The already existing `SEL` function block provides this functionality and has to be used instead.

Example:

Legacy syntax:

```
400010 := 300010 < 300020 ? 300004 : 300005
```

Unity Pro syntax:

```
%MW10 := SEL(%IW10 < %IW20, %IW4, %IW50);
```

With the following, automatically created by the editor:

```
_400010 := SEL(_300010 < _300020, _300004, _300005);
```

Bitwise Shift

Bitwise shift (<<, >>), in contrast to legacy LL984, is not available in Unity Pro ST syntax.

The already existing `SHL` and `SHR` function blocks provide this functionality and have to be used instead.

Example:

Legacy syntax:

```
400010 = 300010U << 3
```

Unity Pro syntax:

```
%MW100W := SHL(%IW10W, 3);
```

With the following, automatically created by the editor:

```
_400100_W := SHL(_300010_W, 3);
```

Function Blocks

In general you can use the function blocks provided for ST language.

In particular the following Unity Pro function blocks replace the legacy function blocks listed below.

Legacy Function Name	Unity Pro Function Name	Argument Type*	Return Type*	Meaning
ABS	ABS	S,U,L,UL,F	S,U,L,UL,F	absolute value
ARCCOS	ACOS	F	F	arc cosine
ARCSIN	ASIN	F	F	arc sine
ARCTAN	ATAN	F	F	arc tangent
COS	COS	F	F	cosine
*: S=INT, U=UINT, L=DINT, UL=UDINT, F=REAL				

Legacy Function Name	Unity Pro Function Name	Argument Type*	Return Type*	Meaning
COSD	COSD	F	F	cosine of degrees
EXP	EXP	F	F	exponent function (power of e) (does not need to be a whole number)
FIX	REAL_TO	F	L	convert floating point to integer (presumes an FP argument)
FLOAT	INT_TO	S, U, L, UL	F	converts integer to floating point (presumes an integer argument)
LN	LN	F	F	natural logarithm (base e)
LOG	LOG	F	F	common logarithm (base 10)
SIN	SIN	F	F	sine of radians
SIND	SIND	F	F	sine of degrees
SQRT	SQRT	F	F	square root
TAN	TAN	F	F	tangent of radians
TAND	TAND	F	F	tangent of degrees
*: S=INT, U=UINT, L=DINT, UL=UDINT, F=REAL				

Example of an LL984 Expression

Legacy syntax:

```
40701 = 40702U + COS(40703UL) * #8.00135F + SIN(40704);
```

Unity Pro syntax:

```
%MW701 := REAL_TO_UINT(WORD_TO_REAL(%MW702W) + COS(WORD_AS_REAL(%MW703W,
%MW704W)) * 8.00135 + SIN(WORD_TO_REAL(%MW704W)));
```

With the following, automatically created by the editor:

```
_400701 := REAL_TO_UINT(WORD_TO_REAL(_400702_W) +
COS(WORD_AS_REAL(_400703_W, _400704_W)) * 8.00135 +
SIN(WORD_TO_REAL(_400704_W)));
```

Section 6.7
Assigning Variables to LL984 Objects

Assigning Variables to LL984 Objects

Introduction

In the LL984 Editor you can assign a variable (like `_400001` or `sym1`) or an address (like %M1) to an FFB pin or to a contact/coil.

If the variable is not declared, the **Create variable?** dialog opens.

If the variable or address assigned is not compatible with the pin or contact/coil, a message will be displayed in the output window after **Build → Analyze**.

Automatic Variable Naming

If you, for example, assign an address (e.g. %MW1) to a pin, the **Create variable?** dialog opens proposing a variable name (e.g. _400001) and a data type (e.g. UINT).

The following rules apply to this automatic proposal, depending on the address of the actual parameter, the pin type and dimension (n) in case of an array type:

Address	Pin Type	Dimension (for array types)	Variable Name
%MW1	UINT	1 (n)	_400001 (_400001_n)
	INT		_400001_I (_400001_In)
	WORD		_400001_W (_400001_Wn)
%IW1	UINT	1 (n)	_300001 (_300001_n)
	INT		_300001_I (_300001_In)
	WORD		_300001_W (_300001_Wn)
%I1	EBOOL	1 (n)	_100001 (_100001_n)
%M1	EBOOL	1 (n)	_000001 (_000001_n)

NOTE: For the most used pin types UINT and EBOOL no type suffix are appended to the proposed variable name.

Use Case 1

Assignment to a pin of type UINT

User-Assigned Address	Variable	Variable Created and Assigned	Variable Type
%MW1	not existing yet	_400001	UINT

NOTE: As the pin type was UINT no type suffix was appended to the variable name.

Use Case 2

Assignment to a pin of type ARRAY[1...n] of UINT

User-Assigned Address	Variable	Variable Created and Assigned	Variable Type
%MW1	not existing yet	_400001_n	ARRAY[1...n] of UINT

Use Case 3

Assignment to a pin of type ANY or ANY_ARRAY_UINT or ANY_ARRAY_BOOL (dynamic array)

The dimension (n) of a dynamic array will be provided by another dimension pin (e.g. LENGTH) of the function block.

This dimension pin will be initialized during block insertion.

The dimension of the array will be calculated based on the dimension pin value entered as actual parameter of the dimension pin.

- If the resulting type is a UINT array:
  dimension = dimension pin value.
- If the resulting type is a Boolean array:
  dimension = dimension pin value * 16.

User-Assigned Address	Variable	Variable Created and Assigned	Variable Type
%MW1	not existing yet	_400001_n	ARRAY[1...n] of UINT

If you modify n on the dimension pin to n2:

User-Assigned Address	Variable	Variable Created and Assigned	Variable Type
%MW1	not existing yet	_400001_n2	ARRAY[1...n2] of UINT
%MW1	already existing	_400001_n2 attributes (like descriptors, comments etc.) are taken over	ARRAY[1...n2] of UINT

Use Case 4

Assignment to a pin of type DDT1

User-Assigned Address	Variable	Variable Created and Assigned	Variable Type
%MW1	not existing yet	_400001_DDT1	DDT1

Memory Consumption

If you create any kind of located variables (elementary type, array or DDT) this will not increase memory consumption on the PLC (except for Upload information).

New Variable Creation

If you assign a different address or modify a dimension pin, a new variable will be created.

The old attributes (except the address) will be overtaken and the old variable will be deleted, if it is not used anywhere else in the program.

Invalid Variables

If you create a located variable with an address range which exceeds the address range defined under **State RAM** of the CPU´s **Configuration** tab, the following message will be displayed after **Build → Analyze**:

```
Topological address index is not valid.
```

There are different ways to solve this issue:

- Click the message displayed in the output window to open the **Data Editor** and adapt the address of the variable.
- Click the message displayed in the output window to open the **Data Editor** and delete the variable and create a new variable with an applicable address range.
- Adapt the address range defined under **State RAM** of the CPU´s **Configuration** tab.

NOTE: If you just create a new variable at the pin of the function block without deleting the previously created, invalid variable the message mentioned above will be displayed again after next **Build → Analyze**.

Example:

Under **State RAM** of the CPU´s **Configuration** tab you entered a value 10000 for **%MW**.

In your program you have an `L_BLKM` function block with `LENGTH` = 10.

At the `SOURCE` pin of the function block you enter following value: `%MW9999`.

This results in creation of the following variable:

```
_409999:ARRAY[1..10] OF UINT@%MW9999
```

As the start address is `%MW9999` and the length is 10, the end address will be `%MW10009`.

This exceeds the address range (`10000`) defined under **State RAM** of the CPU´s **Configuration** tab.

Therefore the message mentioned above will be displayed after **Build → Analyze**.

Section 6.8
Calling Subroutines

Calling Subroutines

Introduction

Calling subroutines is managed by 3 function blocks:

- L9_JSR
- L9_LAB
- L9_RET

For detailed information please refer to the function block descriptions *(see Unity Pro, UnityLL984, Block Library)*.

Up to 1023 subroutines can be established.

Unity Pro does not support interrupt handling with LL984 subroutines.

Restrictions

For the calling subroutines feature the following restrictions apply:

- Only one LL984 segment can be created inside the subroutine (**SR Section**) node of the **MAST** task.
- L9_LAB and L9_RET function blocks can only be placed in an LL984 subroutine network in the **SR Sections**.
- An L9_LAB function block can only be placed in row 1, column 1 of an LL984 subroutine network.

Subroutine Structure

A subroutine is identified by a number (1.. 1023).

Subroutines are called by the L9_JSR function block within an LL984 network (normal or subroutine network).

Two additional function blocks, which are only available in subroutine networks, mark the starting point (L9_LAB function block) and ending point (L9_RET function block) of a subroutine.

A subroutine may span multiple networks.

The L9_RET function block is optional and this explicit ending point of a subroutine can be omitted,

- if the last subroutine network is the last network of the subroutine segment or
- if a new subroutine network (L9_LAB) would entered.

The L9_RET function block or the implicit forms, return from the current subroutine network to the calling L9_JSR function block with an OK status.

Nesting

A subroutine may call other subroutines or recursively call itself up to 100 times.

If the maximum nesting level is reached, an implicit return is performed and the `ERR` output of the `L9_JSR` function block is set.

A nesting level problem message is propagated to the (99) cascaded `L9_JSR` function blocks.

Further cascaded `L9_JSR` function blocks, which are being executed after a nesting level problem, do not succeed to call a subroutine and the `ERR` output is being set until the nesting level stack has been unwound.

NOTE: The message is only visible at those `L9_JSR` function blocks, contributing to the problem. A dedicated problem treatment is possible on each calling level. Subsequent `L9_JSR` function blocks of a segment are not influenced by this problem.

Please also refer to the `L9_JSR` function block description *(see Unity Pro, UnityLL984, Block Library)*.

Section 6.9
Entering Comments

Entering Comments

Introduction

Comments can be placed in LL984 sections in the form of text objects.

Text objects can overlap other objects.

The size of the object, depending on the size of the text, can be extended vertically and horizontally to fill further grid units.

The same rules apply for entering text and navigating within text objects as for editing ASCII text in standard text editors. (To create a line break in text objects you must press the **Ctrl+Enter** key combination).

NOTE: Keep in mind that each change to a comment (e.g. change to a comment text, change to the size of the text object) makes it necessary to recreate the section involved (**Build → Build Changes**).

Selecting the Text Object

You have the following options to select text objects:

- Use the **Edit → New → Comment** menu command.
- Use the **Comment** menu from the context menu.
- Press the **F8** key.
  or
- Select the  symbol.

The active placement mode for text objects is indicated by the  cursor symbol.

Placing Text Objects

Step	Action
1	Select the text object.
2	Click the target cell in the LL984 section. or Use the **Arrow** keys to move the gray field to the target position and press **Enter**. **Result:** The text object is inserted. Example: 
3	Enter the comment.
4	Confirm the text entered with: ● Clicking with the mouse outside of the text object or ● Pressing the **Enter** key.
5	To insert further text objects: ● Click the target cell in the LL984 section. or Use the **Arrow** keys to move the gray field to the target position and press **Enter**. ● To return to selection mode: Press the **Esc** key. ● To insert other objects: Select the object you wish to insert.

Chapter 7
Programming Languages Data Selection Dialog

Data Properties

Introduction

The anchored window *(see Unity Pro, Operating Modes)* **Data Properties** is used to display and modify the variables and FFB properties.

The **Data Properties** window can be called up

- using the Data Editor *(see Unity Pro, Operating Modes)*.
  a. Select one or several lines in the Data Editor.
  b. Select **Properties** from the context menu.
- the programming language sections
  a. Select one or several element(s) in the programming language section.
  b. Select **Data Properties** from the context menu or press **CTRL+Enter**.
- the tab **I/O Objects** of a module
  a. Select one or several lines in the **Address/Name/Type** table.
  b. Select **Data Properties** from the context menu.

How the window behaves and the options in the **Data Properties** window depend on whether the window was opened using the Data Editor or the Programming Languages section.

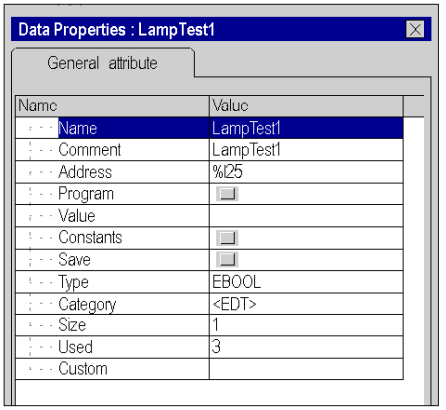
NOTE: If more than one object is selected, only the common data for all the selected objects is displayed. If this data is modified, the modification applies to all the selected objects. In this way it is possible to assign the same comment to different variables or assign the same initial value, see also: *Unity Pro, Operating Modes*

⚠ CAUTION
LOSS OF DATA Before changing a data in the **Data Properties** window, ensure that the change is appropriate not only to the selected instance of this data, but also to all other instances of this data. **Failure to follow these instructions can result in injury or equipment damage.**

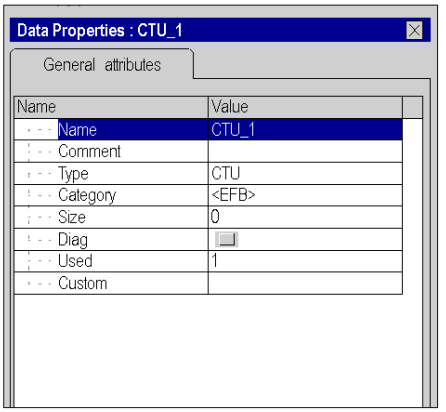
Structure of the Window

The contents of the window depend on the selected object:

Window **Data Properties** for elementary variables:



Window **Data Properties** for function blocks:



Available Attributes

The list of available attributes is the same as the **Data Editor** columns. Refer to the Data Editor columns *(see page 15)* for more information.

Chapter 8
Search/Replace Tool

Subject of this Chapter

This chapter presents the **Search / Replace** tool and how to use its main functions:

- the **Search** function is used to look for data,
- the **Replace** function is used to replace data.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Overview of the Search / Replace tool	130
General Features for Search/Replace	132
Searching for Data	133
Replacing Data	134
Search Toolbar	135
Data Search Dialog	136
Cross-References (Search Results)	138
Cross-References (Replace)	141
Tracing Variables / Addresses	143

Overview of the Search / Replace tool

Introduction

The **Search / Replace** tool provides you with the following functions:
- the **Search** function,
- the **Replace** function.

Search Function

This function lets you **look for objects** such as:
- **instances** of elementary data types (EDT),
- **instances and types** of derived data (DDT / IODDT),
- **instances and types** of function block data (EFB / DFB),
- **instances and types** of data relating to Sequential Function Charts (SFC),
- **topological addresses** (`%M100`).

The search is carried out in the following areas of the project:
- sections that make up the application program,
- animation tables,
- operator screens,
- configuration.

Replace Function

This function lets you **replace objects** such as:
- **instances** of elementary data types (EDT),
- **instances** of derived data types (DDT / IODDT),
- **instances** of function block data (EFB / DFB),
- **topological addresses** (`%M100`).

The replacement objects must be declared, except for direct addressing data (`%M`, `%MW`, etc.) and their type must correspond to that of the objects being replaced.

The replacement can done on the search results and initiated at the result window.

Accessing the Data Search Dialog

To open the **Data Search** dialog

- click the **Data Search**  button in the toolbar or
- use the **Tools → Search / Replace** menu (**Alt+5**).

Predefined Search

To start a predefined search

- from the **Data Editor** or
- from a language editor window,

select the object whose references you want to see and use **Initialize Search** from the context menu.

Another way is to select a variable in a language editor window. Doing so, the list box of the search toolbar *(see page 135)* contains the variable name. Press **Enter** to start searching. You can also use 2 special buttons from the toolbar to search for reading/writing references *(see page 135)*.

General Features for Search/Replace

Copy / Paste

You can use copy and paste in the following instances:

- copying a data item from any given editor in the programming software and pasting it into an entry field in the window,
- copying a data item from the window and pasting it into any given editor in the programming software,
- copying / pasting a data item within a window.

Drag-and-Drop

You can use drag-and-drop for search.

Drag a data item from any given editor in the programming software and drop it at the end of the list of displayed references; the corresponding tree is created.

Protection

The **Search** function is always available. Protection applies to:

- the references in the protected sections of the program, which are not displayed,
- the references in the protected sections of the DFBs, which are not displayed.

The **Replace** function complies with the protection applied to the different areas of the project:

- if the application field is in read-only, the **Replace** button is not available,
- if certain references are protected, they can not be selected for replacement.

Online Mode

The use and behavior of **Search / Replace** in online mode are identical to the use and behavior in offline mode.

Save Context

When you close the Search / Replace window, the following data is saved:

- the position and size of the window,
- the selection of search parameters,
- the 10 most recent entries in the drop-down menu,
- the current view (structural/functional and full/shortened/flat),
- the contents of the Unity Pro **Search / Replace** output window.

Searching for Data

Searching via Data Search Dialog

To search data using the **Data Search** dialog, you must perform the following actions:

Step	Action
1	Open the **Data Search** dialog ● by clicking the **Data Search** button  in the toolbar or ● by using the **Tools → Search / Replace** menu (**Alt+5**) and pressing the **Data Search** button in the **Cross-References** window *(see page 138)*. **Result**: The **Data Search Dialog** is displayed.
2	Specify the search options in the **Data Search Dialog** *(see page 136)*.
3	Click the **Search** button. **Result**: The search results are displayed in the **Cross-References** window *(see page 138)*.

Predefined Search

To start a predefined search

- from the **Data Editor** or
- from a language editor window,

select the object whose references you want to see and use **Initialize Search** from the context menu.

Another way is to select a variable in a language editor window. Doing so, the list box of the search toolbar *(see page 135)* contains the variable name. Press **Enter** to start searching. You can also use 2 special buttons from the toolbar to search for reading/writing references *(see page 135)*.

Replacing Data

To access the data replace function, you must perform the following actions:

Step	Action
1	After searching for data *(see page 133)* the search results are displayed in the **Cross-References** dialog *(see page 138)*.
2	In the **Cross-References** dialog click the  button to display the **Replace with** area of the dialog.
3	Enable the check boxes for those references of the search results which should be replaced.
4	• Enter the replacement data (only data compatible with the data being searched for is tolerated) • or click the **Down** arrow to select the replacement data out of a list providing the last 10 entries • or click the **...** button to open a **Data Selection** dialog to select the instance or data to be replaced (only the instances compatible with the instance being searched for are displayed)
5	Click the **Replace** button to start the replacement operation in the preselected **Scope**. Please refer to **Data Search** dialog *(see page 136)*.

Search Toolbar

Introduction

The main toolbar of Unity Pro provides a **Data Search** button with a list box beside it to select or to enter a search string.



Start Searching

To start searching the selected or entered search string press **Enter**.

The default search options are used.

The options can be changed by pressing the **Data Search** button left to the list box instead of pressing **Enter**.

The search results are displayed in the **Cross-References** dialog *(see page 138)* and a jump to the first occurrence of the searched data is done.

Data Search Dialog

To open the **Data Search** dialog *(see page 136)* click the binoculars symbol.

Searching for Reading/Writing References

To search for reading/writing references only, you can use

- the **Data Search** dialog *(see page 136)* or
- two special buttons from the toolbar (see below).

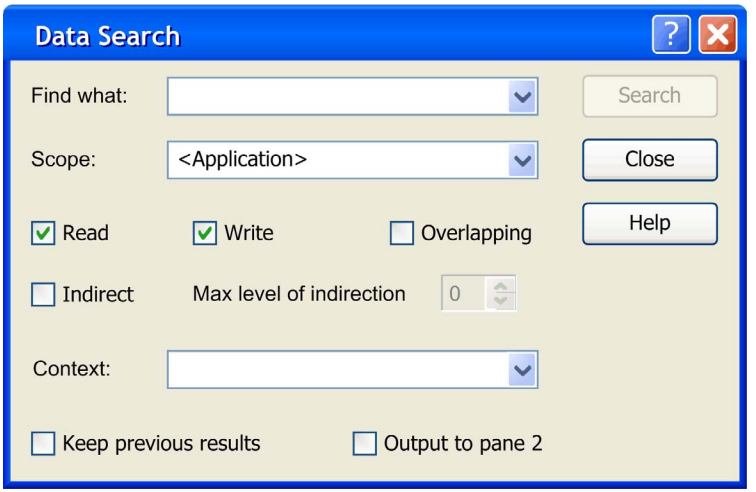
Button	Description
	Click this button to start searching for reading references only. The current search options, defined in the **Data Search** dialog, are used but the **Write** option is ignored.
	Click this button to start searching for writing references only. The current search options, defined in the **Data Search** dialog, are used but the **Read** option is ignored.

Using one of these 2 buttons initiates a direct jump to the first found occurrence of the reading/writing reference (for example, into the containing network).

If instead of a direct jump to the first found occurrence of the reading/writing reference the **Data Editor** is opened, either deactivate the Overlapping *(see page 136)* option or click the **Go to next reference** button *(see page 138)*.

Data Search Dialog

Representation



Description

Elements of the **Search** tab

Element	Description
Find what	Enter the complete variable name you want to search for. Click the **Down** arrow to display a list providing the last 10 search strings. Click the **...** button to open a **Data Selection** dialog to select the object you want to search for.
Scope	Select the scope where you want to search. The different scopes are: ● application ● a specific DFB of the project ● all scopes (application + all DFBs)
Search button	Starts the search and opens search results window (**Cross-References**).
Close button	Closes the dialog.
Help button	Opens the Help for this dialog.
Read	Limits the search to reading references.
Write	Limits the search to writing references.

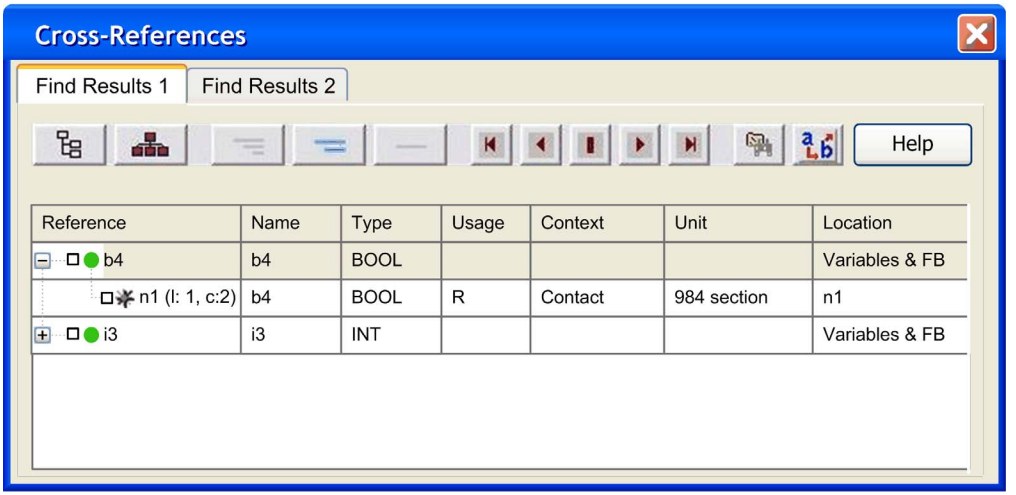
Element	Description
Overlapping	Activate this check box to search for all complete and partial references.
Indirect	Activate this check box to search for sub-elements and direct parents of structured variables and arrays.
Max level of indirection	Defines the maximum level of indirect elements (sub-elements/parents) to be displayed.
Context	Limits the search to a specific context. • **Contact**: variable attached to a contact • **Coil**: variable attached to a LD/LL984 coil • **SFC Object**: variable used with an SFC object • **FFB / Subroutine**: variable passed to an FFB • **ST / IL Statement**: variable used with an ST/IL statement The context is displayed as an optional column in the search results.
Keep previous results	Activate this check box to keep the previous search results. Otherwise, the search results window (**Cross-References**) is cleared before the new search results are displayed.
Output to pane 2	Deactivate this check box to display the search results in the **Find Result 1** tab of the search results window (**Cross-References**). Activate this check box to display the search results in the **Find Result 2** tab of the search results window (**Cross-References**).

Cross-References (Search Results)

Introduction

The **Cross-References** search results dialog displays the results of a search.

Representation



Description

Elements of the **Cross-References** dialog

Element	Description
Find Result 1 tab **Find Result 2** tab	Depending on the respective check box activated in the **Data Search** dialog *(see page 136)*, the search results are displayed in one of the 2 tabs.
	Displays the search results in a similar manner to the structural view in the **Project Browser**.
	Displays the search results in a similar manner to the functional view in the **Project Browser**.
	Displays the search results in **Full** form. ● Data declaration ● Path ● Data usage

Element	Description
	Displays the search results in **Shortened** form. ● Data declaration ● Data usage
	Displays the search results in **Flat** form. ● Data usage
	Selects the previous reference of the search results.
	Selects the next reference of the search results.
	Jumps directly to the currently selected found occurrence of the reference.
	Jumps directly to the previous found occurrence of the reference (for example into the containing network).
	Jumps directly to the next found occurrence of the reference.
	Opens the **Data Search** dialog.
	Displays/hides the **Replace with** area of the dialog *(see page 141)*.
Reference	This column displays the name of the instance or type (DDT/IODDT/EFB/DFB) of the data searched for. The details displayed depend on the display settings described above (**Full**, **Shortened**, **Flat**).
Type	This column displays ● type and reference of the data searched for, ● programming language used in the sections, subroutines, events, etc. that contain references to the data searched for.
Usage	This column displays ● the nature of the references to the data searched for (read only, write only, read / write), ● the data search area.
Context	Specific context as defined in the Data Search Dialog *(see page 136)*.
Unit	This column displays the unit of the **Location**.
Location	This column displays the location of the data searched for in a short form.

Sorting Order

When the search results are displayed in **Full** or **Shortened** form sorting can only be done by **Reference** or **Type**.

When the search results are displayed in **Flat** form sorting can also be done by reference related columns (**Usage**, **Context**, **Unit** and **Location**).

The search results are first sorted by their path (Program, Task, Segment, Section/Network).

The search results of sections/networks are sorted by a second key, depending on the kind of the section/network.

- References within IEC language sections (FBD, LD, SFC, ST, IL) are sorted by row and column.
- References within LL984 networks are sorted by column and row.

You can re-sort the search results by clicking on a specific column.

Re-sorting by multiple keys is not supported.

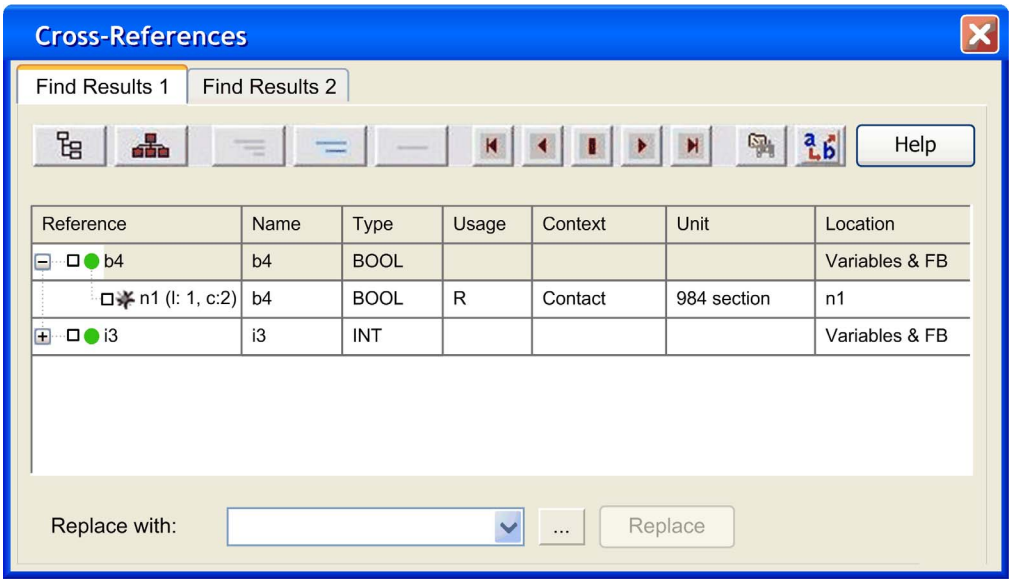
Cross-References (Replace)

Introduction

The **Cross-References** (Search Results) dialog *(see page 138)* displays the results of a search.

To display the **Replace with** area of the dialog use the  button.

Representation



Description

Elements of the **Replace with** area of the dialog

Element	Description
Replace with	To display this area of the dialog use the  button. ● Enable the check boxes for those references of the search results which should be replaced. ● Enter the replacement data (only data compatible with the data being searched for is tolerated), ● or click the **Down** arrow to display a list providing the last 10 entries, ● or click the **...** button to open a **Data Selection** dialog to select the instance or data to be replaced (only the instances compatible with the instance being searched for are displayed).
Replace button	Starts the replacement operation in the preselected **Scope**. Please refer to **Data Search** dialog *(see page 136)*.

Tracing Variables / Addresses

Introduction

Using the tracing feature, you can locate the occurrences of a variable/address in an application.

Tracing

Step	Action
1	Select an object in a language editor window. **Result**: The address attached to the object is displayed in the edit field of the Search Toolbar *(see page 135)*.
2	Start a search either ● by pressing the **Enter** key or ● by means of one of the 2 special buttons next to the edit field of the Search Toolbar (Searching for Reading/Writing References *(see page 135)*). **Result**: ● The search results are displayed in the **Cross-References** dialog *(see page 138)*. ● A direct jump is initiated to the first found occurrence of the reading/writing reference (for example into the containing network). ● A new automatic **Search_"<Search String>"** bookmark is created (to be able to jump back later on). This bookmark is inserted into the bookmarks list like any other manually created bookmark. Automatically created bookmarks *(see page 57)* are displayed in gray and can not be renamed.
3	Double-click a search result, displayed in the **Cross-References** dialog, to jump to the respective read/write location.
4	With the bookmarks feature you can mark one or more of the shown locations in the language editor. These bookmarks can be used later on to jump to from the bookmarks window.
5	From the bookmarks window you can also select the automatic **Start of Search** bookmark for going back to the original starting position of the search. **NOTE:** You can delete automatically generated bookmarks via the bookmarks window.

Part III
Debugging and Adjustment

Chapter 9
Viewing and Adjusting Variables

Overview

This chapter describes the specifics of the LL984 Editor referring to viewing and adjusting Variables.

For standard features please refer to *Viewing and Adjusting Variables (see Unity Pro, Operating Modes)*.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Force Mode	148
Instruction Editor	154

Force Mode

Preset Force Values in Offline Mode

The **Modification Values** and **Force Values** for variables can be preset in offline mode via animation tables. This information can be transferred to PLC later.

Presetting values can also be used for multiple modification and forcing of Boolean variables *(see Unity Pro, Operating Modes)*.

Preset Force Values in Project Information

The preset force values of variables are included in the

- download (**Transfer Project to PLC**) information of a project
- upload (**Transfer Project from PLC**) information of a project.

The **Modification Value**/**Force Value** attribute of a variable is part of the animation table information.

Include in Upload Information

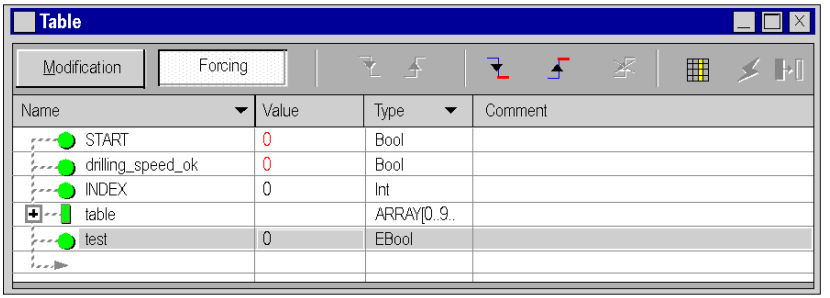
In the properties dialog of an animation table you can activate/deactivate an additional property (**Include in Upload Information**).

As a default this property is activated.

This check box in the property dialog of an animation table is enabled only, if **Animation tables** is activated via **Project Settings → General → PLC embedded data → Upload information**.

Implementing Forcing Mode

Press the **Force** button in the **Operating Mode** area:



This mode is only available for variables responding to these conditions:

- The variable is an `EBool` type.
- The forcing attribute is validated in the variable editor.
- The variable is:
  - a located variable, or
  - for Modicon M580 with a firmware version greater or equal to 2.10, the variable can be not located in an EDT or in a Device DDT,
  - for Modicon Quantum 140 CPU 651 50, 140 CPU 651 60, 140 CPU 652 60,140 CPU 658 60, 140 CPU 670 60, 140 CPU 671 60, 140 CPU 672 60, 140 CPU 672 61, 140 CPU 678 61, with a firmware version greater or equal to 3.0 the variable can be not located but only in a Device DDT (X80 modules in EIO drops).

NOTE: %SW108 *(see Unity Pro, System Bits and Words, Reference Manual)* is not updated by the not located forced EBOOL.

NOTE:
Analog I/O variables forcing is managed in a different way:

- On analog Device DDT *(see Modicon M340 with Unity Pro, Analog input/output modules, User manual)*, a set of sub fields (for example EIO2_d1_r0_s1_A-MI0800.ANA_CH_IN[0].ANA.FORCE_CMD) allows to force analog I/Os with a predefined value.
- For Modicon M580 and M340 PLCs the analog I/Os variables can be forced in the program.

NOTE: The **FORCED I/O** led of the M580 will switch to 1 when forced is active.

NOTE: With the animation table, it is not possible to set or reset the bit associated to step S.x.

Forcing input and output values in a running controller can have serious consequences to the operation of a machine or process. Only those who understand the implications in the controlling logic, and who understand the consequences of forced I/O on the machine or process, should attempt to use this function.

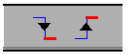
⚠ WARNING
UNINTENDED EQUIPMENT OPERATION You must have prior knowledge of the process and the controlled equipment and the modified behavior in Unity Pro before attempting to force I/O or numeric values to memory locations. **Failure to follow these instructions can result in death, serious injury, or equipment damage.**

NOTE:
Before forcing outputs, understand that forced outputs (%M) exhibit different behavior with Modsoft/NxT/Concept than Unity Pro.

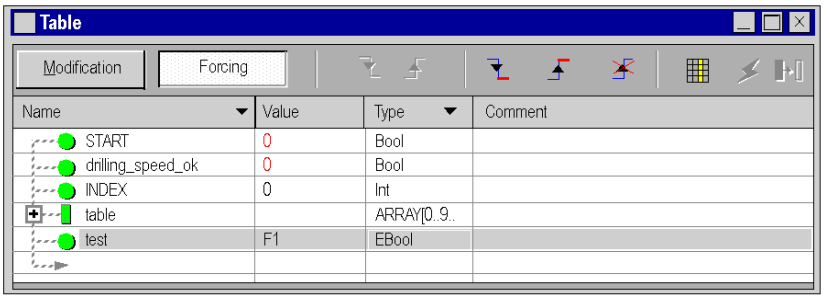
- With Modsoft/NxT/Concept you **cannot** force outputs when the Memory Protect switch of the Quantum CPU is set to the "On" position.
  With Unity Pro you **can** force outputs even when the Memory Protect switch of the Quantum CPU is set to the "On" position.
- With Modsoft/NxT/Concept forced outputs **maintain** their values following a cold start.
  With Unity Pro forced outputs **lose** their disabled state following a cold start.

Forcing a Variable

To force a localized Boolean variable, carry out the following actions:

Step	Action
1	Use the mouse to select the Boolean variable.
2	Press one of the buttons  corresponding to the desired value, or from the contextual menu, execute the **Force to 0** or **Force to 1** commands. To force several variables simultaneously, see Forcing several located variables *(see Unity Pro, Operating Modes)*.

When a variable is forced, the letter **F** is displayed in front of the value:



Forcing input and output values in a running controller can have serious consequences to the operation of a machine or process. Only those who understand the implications in the controlling logic, and who understand the consequences of forced I/O on the machine or process, should attempt to use this function.

⚠ WARNING
UNINTENDED EQUIPMENT OPERATION You must have prior knowledge of the process and the controlled equipment and the modified behavior in Unity Pro before attempting to force I/O or numeric values to memory locations. **Failure to follow these instructions can result in death, serious injury, or equipment damage.**

NOTE:
Before forcing outputs, understand that forced outputs (%M) exhibit different behavior with Modsoft/NxT/Concept than Unity Pro.

- With Modsoft/NxT/Concept you **cannot** force outputs when the Memory Protect switch of the Quantum CPU is set to the "On" position.
  With Unity Pro you **can** force outputs even when the Memory Protect switch of the Quantum CPU is set to the "On" position.
- With Modsoft/NxT/Concept forced outputs **maintain** their values following a cold start.
  With Unity Pro forced outputs **lose** their disabled state following a cold start.

Unforcing a Variable

To unforce a located Boolean variable, carry out the following actions:

Step	Action
1	Use the mouse to select the Boolean variable.
2	Press the  . button or execute the **Unforce** command from the contextual menu. To unforce several variables simultaneously, see Unforcing several located variables *(see Unity Pro, Operating Modes)*

When a variable is unforced, the letter **F** no longer appears in front of the value.

Creating an Offline Forcing Animation Table

To create a new animation table of forced bits from scratch, carry out the following actions:

Step	Action
1	Create a new (permanent) animation table via the **Project Browser**.
2	Enter the memory bits you want to force either as single bits (like %M1) or as a bit array (like M1:100).
3	Press the **Force** button.
4	Press the **Swaps to multiple mode**  button.
5	Enter forcing values **0** or **1** in the **Force value** column.

To create a new animation table of existing forced bits in the PLC, carry out the following actions:

Step	Action
1	There are 3 ways to open the **Forced bits** animation table: ● In the status bar click the red **F**. ● In the **Project Browser** right-click **Animations Tables** and select **Open Forced Bits Table** from the context menu. ● Via **Tools → PLC Screen** open the **PLCScreen** dialog. In the **Information** tab under **APPLICATION → MISCELLANEOUS** click the **FORCED BITS** line.
2	Since this animation table is temporary only, copy and paste the animation table via context menu of the **Project Browser**.
3	Make the copied animation table permanent (using the **Temporary table** button ).
4	In the **copied Forced bits** animation table press the **Force** button.
5	Press the **Swaps to multiple mode**  button.
6	Press the **Populates multiple values using existing forced bits**  button. **Result**: The **Force value** column is populated.

Build/Build changes and **Save** your application.

By building, the values entered in the **Force value** column will be included in the upload information which is downloaded into the PLC.

Forcing by Use of an Offline Forcing Animation Table

To force values before starting the PLC, carry out the following actions:

Step	Action
1	Open an existing application (**.STU*, **.STA* or an uploaded application).
2	Connect to the PLC.
3	Open the manually created/copied **Forced bits** animation table (see *Creating an Offline Forcing Animation Table* above).
4	Press the **Force** button to enter forcing mode.
5	Press the **Swaps to multiple mode**  button.
6	Press the **Execute multiple**  button to force the bits from the preset **Force value** column.
7	Now you can start your application.

Forcing input and output values in a running controller can have serious consequences to the operation of a machine or process. Only those who understand the implications in the controlling logic, and who understand the consequences of forced I/O on the machine or process, should attempt to use this function.

⚠ WARNING

UNINTENDED EQUIPMENT OPERATION

You must have prior knowledge of the process and the controlled equipment and the modified behavior in Unity Pro before attempting to force I/O or numeric values to memory locations.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

NOTE:
Before forcing outputs, understand that forced outputs (%M) exhibit different behavior with Modsoft/NxT/Concept than Unity Pro.

- With Modsoft/NxT/Concept you **cannot** force outputs when the Memory Protect switch of the Quantum CPU is set to the "On" position.
  With Unity Pro you **can** force outputs even when the Memory Protect switch of the Quantum CPU is set to the "On" position.
- With Modsoft/NxT/Concept forced outputs **maintain** their values following a cold start.
  With Unity Pro forced outputs **lose** their disabled state following a cold start.

Instruction Editor

Introduction

For every Unity Pro FFB you can use the Unity Pro animation tables during runtime. (Select a function block and use **Initialize Animation Table** from the context menu. Please refer to *Animation Tables (see Unity Pro, Operating Modes)*.

But since special FFBs of the UnityLL984 Block Library have parameters of array type there will be no detailed information displayed for these parameters.

So for most of the FFBs of the UnityLL984 Block Library the LL984 editor provides another means of animation called **Instruction Editor**.

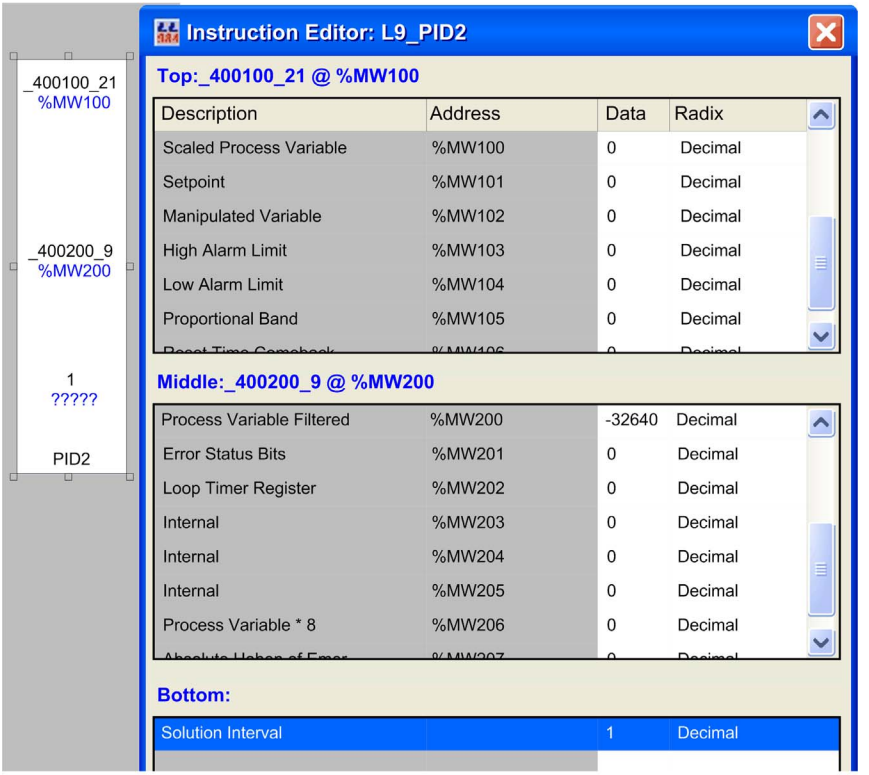
NOTE: For the FFBs of the UnityLL984 Block Library that are not supported by the **Instruction Editor** see table below.

Using the Instruction Editor

To open the **Instruction Editor** for an FFB during runtime, select the FFB and use **Show instruction** from the context menu.

Representation

Representation of the **Instruction Editor**



Items of the **Instruction Editor**

Item	Description
Table Header	displays <Pin name>:<Variable name>@<Address> • pin name of the respective function block node • name of the connected variable
Description	describes the elements of the array
Address	displays the address of the array element in case of located variables In case of unlocated variables this field is empty. In either case the corresponding data column field can be animated/modified.
Data	displays the actual value of the array element The value can be edited by clicking this field.

Item	Description
Radix	allows you to select the displayed data type format Available formats: ● Binary ● Decimal ● Floating Point ● HexaDecimal ● BCD ● ASCII ● Signed Integer ● 32 Bit Integer

Changing the Data Type

If for example an element of the table is defined as of type floating point or 32 bit integer, the next element is shown as disabled.

For some elements you can change the predefined data type.

For example showing a decimal also as binary.

Changing from floating point to decimal will enable the next decimal row.

Changing back to floating point will disable the next decimal row.

Bit Information

For certain memory words, bit information is also available.

In this case positioning the mouse over the **Data** column of such a memory word will show information for the bits, which may be specially useful when **Binary** is selected as data type format (**Radix**).

FFBs Not Supported by Instruction Editor

The following FFBs are not supported by the **Instruction Editor**.

Family	Not Supported FFBs
CONTL_COMM	L9_ID L9_IE
MISC	L9_LAB L9_RET

Appendices



Appendix A
LL984 operating modes

LL984 operating modes

LL984 operating modes

To detect the first cycle in RUN, the recommended solution is to use **%S21** system bit.

Index

B

bookmarks
graphical languages, *53*

C

coils
LL984 editor, *59*
contacts
LL984 editor, *59*, *83*
creating a program
LL984 editor, *59*, *60*

E

equation network blocks
LL984 editor, *59*, *112*

F

FBD editor
search/replace function, *129*
function blocks
LL984 editor, *59*, *101*

G

general functions
LL984 editor, *21*
general restrictions
LL984 editor, *21*
graphical languages
bookmarks, *53*

I

IL editor
search/replace function, *129*
instruction editor
LL984 editor, *154*

L

layout
LL984 editor, *21*
LD editor
search/replace function, *129*
links
LL984 editor, *59*, *106*
LL984 editor
assigning variables, *59*, *119*
coils, *59*, *93*
contacts, *59*, *83*
creating a program, *59*, *60*
creating networks and sections, *33*
equation network blocks, *59*, *112*
function blocks, *59*, *101*
general functions, *21*
general restrictions, *21*
instruction editor, *154*
layout, *21*
links, *59*, *106*
modifying networks and sections, *33*
network display, *59*, *71*
options, *45*
project settings, *45*
scheduling networks and sections, *33*
search/replace function, *129*
subroutines, *59*, *123*
toolbar, *21*

N

network and section creation
LL984 editor, *33*
network and section modification
LL984 editor, *33*
network display
LL984 editor, *59*, *71*
networks and sections scheduling
LL984 editor, *33*

O

options
LL984 editor, *45*

P

project settings
LL984 editor, *45*

S

search/replace function, *129*
SFC editor
search/replace function, *129*
ST editor
search/replace function, *129*
subroutines
LL984 editor, *59*, *123*

T

toolbar
LL984 editor, *21*

V

variables assignment
LL984 editor, *59*, *119*